

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Semana 14 :Desarrollo Ejercicios

CURSO:

- Base de Datos II

DOCENTE:

- Fernández Bejarano Raúl Enrique

ESTUDIANTE:

- Rosales Crispín Aristóteles Onassis

Ciclo:

- 5°SEMESTRE

HUANCAYO- JUNIN
2026

TEMA 1. CREACIÓN DE JOBS Y ALERTS CON SQL SERVER AGENT

Proyecto 1: Enunciado: La Oficina de Grados y Títulos requiere que todos los días a las **08:00 AM** se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.

Código:

```
-- =====
-- PROYECTO 1: Reporte diario de trámites registrados
-- Ejecución: Todos los días a las 08:00 AM
-- =====

USE GradosTitulos;
GO

-- 1. CREAR TABLA DE AUDITORÍA PARA JOBS
IF OBJECT_ID('Auditoria.AuditoriaJobs', 'U') IS NULL
BEGIN
    IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Auditoria')
        EXEC ('CREATE SCHEMA Auditoria');

    CREATE TABLE Auditoria.AuditoriaJobs (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        JobName NVARCHAR(128) NOT NULL,
        StepName NVARCHAR(128) NULL,
        FechaEjecucion DATETIME2 DEFAULT SYSDATETIME(),
        Estado VARCHAR(20) NOT NULL, -- EXITOSO, FALLIDO, EN_EJECUCION
        Mensaje NVARCHAR(MAX) NULL,
        DuracionMs INT NULL,
        RegistrosAfectados INT NULL,
        Detalles XML NULL
    );

    -- Índice para búsquedas rápidas
    CREATE NONCLUSTERED INDEX IX_AuditoriaJobs_JobName_Fecha
    ON Auditoria.AuditoriaJobs (JobName, FechaEjecucion DESC);

    PRINT 'Tabla Auditoria.AuditoriaJobs creada exitosamente.';
END
GO

-- 2. PROCEDIMIENTO PARA REGISTRAR EJECUCIÓN DE JOBS
CREATE OR ALTER PROCEDURE Auditoria.SP_RegistrarEjecucionJob
    @JobName NVARCHAR(128),
    @StepName NVARCHAR(128) = NULL,
    @Estado VARCHAR(20),
    @Mensaje NVARCHAR(MAX) = NULL,
    @RegistrosAfectados INT = NULL,
    @Detalles XML = NULL,
    @IdRegistro INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Auditoria.AuditoriaJobs (
        JobName,
        StepName,
        Estado,
        Mensaje,
        RegistrosAfectados,
        Detalles
    )
```

```

VALUES (
    @JobName,
    @StepName,
    @Estado,
    @Mensaje,
    @RegistrosAfectados,
    @Detalles
);

SET @IdRegistro = SCOPE_IDENTITY();

-- Actualizar duración (aproximada)
DECLARE @Duracion INT;
SELECT @Duracion = DATEDIFF(MILLISECOND, FechaEjecucion, SYSDATETIME())
FROM Auditoria.AuditoriaJobs
WHERE Id = @IdRegistro;

UPDATE Auditoria.AuditoriaJobs
SET DuracionMs = @Duracion
WHERE Id = @IdRegistro;

END
GO

-- 3. PROCEDIMIENTO DEL JOB: Reporte diario de trámites
CREATE OR ALTER PROCEDURE Auditoria.SP_ReporteDiarioTramites
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @IdRegistro INT;
    DECLARE @FechaInicio DATE = CAST(DATEADD(DAY, -1, GETDATE()) AS DATE);
    DECLARE @FechaFin DATE = CAST(GETDATE() AS DATE);
    DECLARE @Mensaje NVARCHAR(MAX);
    DECLARE @Detalles XML;

    BEGIN TRY
        -- Registrar inicio de ejecución
        EXEC Auditoria.SP_RegistrarEjecucionJob
            @JobName = 'ReporteDiarioTramites',
            @StepName = 'Generar reporte diario',
            @Estado = 'EN_EJECUCION',
            @IdRegistro = @IdRegistro OUTPUT;

        -- Generar reporte
        DECLARE @TotalTramites INT;
        DECLARE @PorTipo TABLE (Tipo VARCHAR(60), Cantidad INT);
        DECLARE @PorEstado TABLE (Estado VARCHAR(40), Cantidad INT);

        -- Total de trámites del día anterior
        SELECT @TotalTramites = COUNT(*)
        FROM Tramite.Tramite
        WHERE CAST(FechaInicio AS DATE) = @FechaInicio;

        -- Detalle por tipo de trámite
        INSERT INTO @PorTipo (Tipo, Cantidad)
        SELECT
            tt.Descripcion AS Tipo,
            COUNT(*) AS Cantidad
        FROM Tramite.Tramite t
        JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite
        WHERE CAST(t.FechaInicio AS DATE) = @FechaInicio
        GROUP BY tt.Descripcion
    
```

```

ORDER BY Cantidad DESC;

-- Detalle por estado
INSERT INTO @PorEstado (Estado, Cantidad)
SELECT
    et.Descripcion AS Estado,
    COUNT(*) AS Cantidad
FROM Tramite.Tramite t
JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite =
et.IdEstadoTramite
WHERE CAST(t.FechaInicio AS DATE) = @FechaInicio
GROUP BY et.Descripcion
ORDER BY Cantidad DESC;

-- Construir mensaje
SET @Mensaje = 'REPORTE DIARIO DE TRÁMITES' + CHAR(13) + CHAR(10) +
'Fecha del reporte: ' + CONVERT(VARCHAR, @FechaInicio,
103) + CHAR(13) + CHAR(10) +
'Total de trámites registrados: ' + CAST(@TotalTramites
AS VARCHAR) + CHAR(13) + CHAR(10) +
'-----' +
CHAR(13) + CHAR(10) +
'DETALLE POR TIPO DE TRÁMITE:' + CHAR(13) + CHAR(10);

SELECT @Mensaje = @Mensaje +
' - ' + Tipo + ': ' + CAST(Cantidad AS VARCHAR) + CHAR(13) +
CHAR(10)
FROM @PorTipo;

SET @Mensaje = @Mensaje + CHAR(13) + CHAR(10) +
'DETALLE POR ESTADO:' + CHAR(13) + CHAR(10);

SELECT @Mensaje = @Mensaje +
' - ' + Estado + ': ' + CAST(Cantidad AS VARCHAR) + CHAR(13) +
CHAR(10)
FROM @PorEstado;

SET @Mensaje = @Mensaje + CHAR(13) + CHAR(10) +
'FIN DEL REPORTE - ' + CONVERT(VARCHAR, SYSDATETIME(),
120);

-- Guardar detalles en XML
SET @Detalles = (
    SELECT
        @FechaInicio AS FechaReporte,
        @TotalTramites AS TotalTramites,
        (SELECT * FROM @PorTipo FOR XML AUTO, TYPE) AS PorTipo,
        (SELECT * FROM @PorEstado FOR XML AUTO, TYPE) AS PorEstado
    FOR XML PATH('Reporte')
);

-- Actualizar registro con éxito
UPDATE Auditoria.AuditoriaJobs
SET
    Estado = 'EXITOSO',
    Mensaje = @Mensaje,
    RegistrosAfectados = @TotalTramites,
    Detalles = @Detalles
WHERE Id = @IdRegistro;

-- Imprimir mensaje para el log
PRINT @Mensaje;

```

```

END TRY
BEGIN CATCH
    -- Registrar error
    SET @Mensaje = 'ERROR: ' + ERROR_MESSAGE() +
        ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) +
        ', Línea: ' + CAST(ERROR_LINE() AS VARCHAR) + ')';

    UPDATE Auditoria.AuditoriaJobs
    SET
        Estado = 'FALLIDO',
        Mensaje = @Mensaje
    WHERE Id = @IdRegistro;

    -- Re-lanzar error
    THROW;
END CATCH
END
GO

-- 4. CREAR JOB EN SQL SERVER AGENT
-- NOTA: Este script debe ejecutarse en la base de datos msdb
USE msdb;
GO

-- Verificar si el job ya existe y eliminarlo
IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'ReporteDiarioTramites')
BEGIN
    EXEC sp_delete_job @job_name = 'ReporteDiarioTramites';
    PRINT 'Job existente eliminado.';
END
GO

-- Crear el job
EXEC sp_add_job
    @job_name = 'ReporteDiarioTramites',
    @enabled = 1,
    @description = 'Reporte diario de trámites registrados (08:00 AM)',
    @start_step_id = 1,
    @category_name = 'Reports',
    @owner_login_name = 'sa';
GO

-- Crear el step (paso) del job
EXEC sp_add_jobstep
    @job_name = 'ReporteDiarioTramites',
    @step_id = 1,
    @step_name = 'Generar reporte de trámites',
    @command = 'EXEC GradosTitulos.Auditoria.SP_ReporteDiarioTramites;',
    @database_name = 'GradosTitulos',
    @subsystem = 'TSQL',
    @retry_attempts = 3,
    @retry_interval = 5;
GO

-- Crear el schedule (programación) diaria a las 08:00 AM
EXEC sp_add_jobschedule
    @job_name = 'ReporteDiarioTramites',
    @name = 'Diario_8AM',
    @freq_type = 4, -- Diario
    @freq_interval = 1, -- Todos los días
    @active_start_time = 080000; -- 08:00:00

```

```

GO

-- Asociar el job al servidor
EXEC sp_add_jobserver
    @job_name = 'ReporteDiarioTramites',
    @server_name = '(local)';
GO

PRINT 'Job "ReporteDiarioTramites" creado exitosamente.';
PRINT 'Programación: Todos los días a las 08:00 AM.';
GO

-- 5. VOLVER A LA BASE DE DATOS GRADOSTITULOS
USE GradosTitulos;
GO

-- 6. VERIFICAR LA CREACIÓN DEL JOB
SELECT
    j.name AS JobName,
    j.enabled AS Habilitado,
    j.description AS Descripcion,
    s.name AS ScheduleName,
    s.active_start_time AS HoraInicio,
    CASE s.freq_type
        WHEN 1 THEN 'Una vez'
        WHEN 4 THEN 'Diario'
        WHEN 8 THEN 'Semanal'
        WHEN 16 THEN 'Mensual'
        WHEN 32 THEN 'Mensual relativo'
        WHEN 64 THEN 'Ejecutar al iniciar SQL Agent'
        ELSE 'Otro'
    END AS Frecuencia
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.sysjobschedules js ON j.job_id = js.job_id
JOIN msdb.dbo.sysschedules s ON js.schedule_id = s.schedule_id
WHERE j.name = 'ReporteDiarioTramites';
GO

-- 7. EJECUTAR EL JOB MANUALMENTE (PARA PRUEBA)
-- EXEC msdb.dbo.sp_start_job @job_name = 'ReporteDiarioTramites';
-- GO

-- 8. CONSULTAR HISTORIAL DE EJECUCIÓN
SELECT TOP 10
    Id,
    JobName,
    FechaEjecucion,
    Estado,
    LEFT(Mensaje, 200) AS MensajeResumido,
    DuracionMs,
    RegistrosAfectados
FROM Auditoria.AuditoriaJobs
WHERE JobName = 'ReporteDiarioTramites'
ORDER BY FechaEjecucion DESC;
GO

```

Proyecto 2: Enunciado: Automatizar la actualización semanal del campo **FechaUltMod** para todos los trámites que permanezcan en estado **EN_PROCESO**, registrando la ejecución.

Codigo:-- =====

-- PROYECTO 2: Actualización semanal de FechaUltMod

-- Ejecución: Cada domingo a las 02:00 AM

-- =====

USE GradosTitulos;

GO

-- 1. PROCEDIMIENTO PARA ACTUALIZAR FECHAS DE TRÁMITES EN PROCESO

CREATE OR ALTER PROCEDURE Tramite.SP_ActualizarFechaUltModSemanal
AS

BEGIN

SET NOCOUNT ON;

DECLARE @IdRegistro INT;

DECLARE @RegistrosAfectados INT = 0;

DECLARE @Mensaje NVARCHAR(MAX);

DECLARE @FechaActualizacion DATETIME2 = SYSDATETIME();

DECLARE @ErrorMsg NVARCHAR(4000);

BEGIN TRY

-- Registrar inicio

EXEC Auditoria.SP_RegistrarEjecucionJob

@JobName = 'ActualizacionSemanalFechas',

@StepName = 'Actualizar FechaUltMod',

@Estado = 'EN_EJECUCION',

@IdRegistro = @IdRegistro OUTPUT;

-- Actualizar trámites en estado EN_PROCESO (IdEstadoTramite = 3)

UPDATE Tramite.Tramite

SET

FechaUltMod = @FechaActualizacion,

Observaciones = COALESCE(Observaciones, '') +

CASE WHEN Observaciones IS NOT NULL THEN ' | ' ELSE '' END +

'Actualización automática semanal: ' + CONVERT(VARCHAR,

@FechaActualizacion, 120)

WHERE IdEstadoTramite = 3 -- EN_PROCESO

AND DATEDIFF(DAY, FechaUltMod, @FechaActualizacion) >= 7;

SET @RegistrosAfectados = @@ROWCOUNT;

-- Generar mensaje

SET @Mensaje = 'ACTUALIZACIÓN SEMANAL COMPLETADA' + CHAR(13) + CHAR(10)

+

'Fecha de actualización: ' + CONVERT(VARCHAR,

@FechaActualizacion, 120) + CHAR(13) + CHAR(10) +

'Trámites actualizados: ' + CAST(@RegistrosAfectados AS

VARCHAR) + CHAR(13) + CHAR(10) +

'Estado aplicado: EN_PROCESO (ID=3)' + CHAR(13) +

CHAR(10) +

'Criterio: Trámites con FechaUltMod > 7 días';

-- Actualizar registro de auditoría

UPDATE Auditoria.AuditoriaJobs

SET

Estado = 'EXITOSO',

Mensaje = @Mensaje,

RegistrosAfectados = @RegistrosAfectados,

```

        DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion, SYSDATETIME())
    WHERE Id = @IdRegistro;

    PRINT @Mensaje;

END TRY
BEGIN CATCH
    SET @ErrorMsg = 'ERROR: ' + ERROR_MESSAGE() +
        ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) + ')';

    IF @IdRegistro IS NOT NULL
    BEGIN
        UPDATE Auditoria.AuditoriaJobs
        SET
            Estado = 'FALLIDO',
            Mensaje = @ErrorMsg,
            DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion,
SYSDATETIME())
            WHERE Id = @IdRegistro;
    END

    PRINT @ErrorMsg;
    THROW;
END TRY
END
GO

-- 2. CREAR JOB EN SQL SERVER AGENT
USE msdb;
GO

-- Eliminar job si existe
IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'ActualizacionSemanalFechas')
BEGIN
    EXEC sp_delete_job @job_name = 'ActualizacionSemanalFechas';
    PRINT 'Job existente eliminado.';
END
GO

-- Crear el job
EXEC sp_add_job
    @job_name = 'ActualizacionSemanalFechas',
    @enabled = 1,
    @description = 'Actualización semanal de FechaUltMod para trámites en
EN_PROCESO',
    @start_step_id = 1,
    @category_name = 'Database Maintenance',
    @owner_login_name = 'sa';
GO

-- Crear el step
EXEC sp_add_jobstep
    @job_name = 'ActualizacionSemanalFechas',
    @step_id = 1,
    @step_name = 'Actualizar fechas de trámites',
    @command = 'EXEC GradosTitulos.Tramite.SP_ActualizarFechaUltModSemanal;',
    @database_name = 'GradosTitulos',
    @subsystem = 'TSQL',
    @retry_attempts = 2,
    @retry_interval = 10;
GO

```



```

-- Crear schedule: Cada domingo a las 02:00 AM
EXEC sp_add_jobschedule
    @job_name = 'ActualizacionSemanalFechas',
    @name = 'Semanal_Domingo_2AM',
    @freq_type = 8, -- Semanal
    @freq_interval = 1, -- Domingo (1 = Domingo, 2 = Lunes, etc.)
    @active_start_time = 020000; -- 02:00:00
GO

-- Asociar al servidor
EXEC sp_add_jobserver
    @job_name = 'ActualizacionSemanalFechas',
    @server_name = '(local)';
GO

PRINT 'Job "ActualizacionSemanalFechas" creado exitosamente.';
PRINT 'Programación: Cada domingo a las 02:00 AM.';
GO

```

-- 3. VERIFICAR CREACIÓN

```

USE GradosTitulos;
GO

```

```

SELECT
    j.name AS JobName,
    s.name AS ScheduleName,
    s.active_start_time AS HoraInicio,
    CASE s.freq_interval
        WHEN 1 THEN 'Domingo'
        WHEN 2 THEN 'Lunes'
        WHEN 3 THEN 'Martes'
        WHEN 4 THEN 'Miércoles'
        WHEN 5 THEN 'Jueves'
        WHEN 6 THEN 'Viernes'
        WHEN 7 THEN 'Sábado'
        ELSE 'Otro'
    END AS DiaSemana
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.sysjobschedules js ON j.job_id = js.job_id
JOIN msdb.dbo.sysschedules s ON js.schedule_id = s.schedule_id
WHERE j.name = 'ActualizacionSemanalFechas';
GO

```

-- 4. HISTORIAL DE EJECUCIÓN

```

SELECT TOP 5
    JobName,
    FechaEjecucion,
    Estado,
    RegistrosAfectados,
    LEFT(Mensaje, 150) AS Mensaje
FROM Auditoria.AuditoriaJobs
WHERE JobName = 'ActualizacionSemanalFechas'
ORDER BY FechaEjecucion DESC;
GO

```

Proyecto 3: Enunciado: Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada: Tramite.LogTemporal.

Codigo

```

-- =====
-- PROYECTO 3: Eliminación mensual de registros temporales
-- Ejecución: Primer día de cada mes a las 01:00 AM
-- =====

```

```

USE GradosTitulos;
GO

-- 1. CREAM TABLA TEMPORAL (SI NO EXISTE)
IF OBJECT_ID('Tramite.LogTemporal', 'U') IS NULL
BEGIN
    CREATE TABLE Tramite.LogTemporal (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        IdTramite INT NOT NULL,
        Accion VARCHAR(50) NOT NULL,
        FechaRegistro DATETIME2 DEFAULT SYSDATETIME(),
        Usuario VARCHAR(100) NULL,
        Datos XML NULL
    );
    PRINT 'Tabla Tramite.LogTemporal creada.';
END
GO

-- 2. CREAM TABLA HISTÓRICA (BACKUP DE REGISTROS ELIMINADOS)
IF OBJECT_ID('Tramite.LogTemporalHistorico', 'U') IS NULL
BEGIN
    CREATE TABLE Tramite.LogTemporalHistorico (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        IdOriginal INT NOT NULL,
        IdTramite INT NOT NULL,
        Accion VARCHAR(50) NOT NULL,
        FechaRegistro DATETIME2 NOT NULL,
        Usuario VARCHAR(100) NULL,
        Datos XML NULL,
        FechaEliminacion DATETIME2 DEFAULT SYSDATETIME(),
        EliminadoPor NVARCHAR(128) DEFAULT ORIGINAL_LOGIN()
    );

    -- Índice para búsquedas históricas
    CREATE NONCLUSTERED INDEX IX_LogTemporalHistorico_IdTramite
    ON Tramite.LogTemporalHistorico (IdTramite, FechaEliminacion);

    PRINT 'Tabla Tramite.LogTemporalHistorico creada.';
END
GO

-- 3. PROCEDIMIENTO PARA ARCHIVAR Y ELIMINAR REGISTROS TEMPORALES
CREATE OR ALTER PROCEDURE Tramite.SP_ArchivarYEliminarLogTemporal
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @IdRegistro INT;
    DECLARE @FechaCorte DATE = DATEADD(DAY, -180, GETDATE());
    DECLARE @RegistrosMover INT = 0;
    DECLARE @RegistrosEliminar INT = 0;
    DECLARE @Mensaje NVARCHAR(MAX);
    DECLARE @ErrorMsg NVARCHAR(4000);
    DECLARE @TranName VARCHAR(32) = 'Tran_ArchivarLog';
    DECLARE @RegistrosTabla TABLE (IdOriginal INT);

    BEGIN TRY
        -- Registrar inicio
        EXEC Auditoria.SP_RegistrarEjecucionJob
            @JobName = 'ArchivarLogTemporal',
            @StepName = 'Archivar y eliminar logs > 180 días',

```

```

        @Estado = 'EN_EJECUCION',
        @IdRegistro = @IdRegistro OUTPUT;

-- Iniciar transacción
BEGIN TRANSACTION @TranName;

-- 1. Insertar registros en tabla histórica
INSERT INTO Tramite.LogTemporalHistorico (
    IdOriginal,
    IdTramite,
    Accion,
    FechaRegistro,
    Usuario,
    Datos
)
OUTPUT INSERTED.IdOriginal INTO @RegistrosTabla
SELECT
    Id,
    IdTramite,
    Accion,
    FechaRegistro,
    Usuario,
    Datos
FROM Tramite.LogTemporal
WHERE CAST(FechaRegistro AS DATE) < @FechaCorte;

SET @RegistrosMover = @@ROWCOUNT;

-- 2. Eliminar registros de la tabla temporal (usando los IDs
respaldados)
IF @RegistrosMover > 0
BEGIN
    DELETE FROM Tramite.LogTemporal
    WHERE Id IN (SELECT IdOriginal FROM @RegistrosTabla);

    SET @RegistrosEliminar = @@ROWCOUNT;
END

-- Confirmar transacción
COMMIT TRANSACTION @TranName;

-- Generar mensaje
SET @Mensaje = 'ARCHIVO Y ELIMINACIÓN DE LOG COMPLETADO' + CHAR(13) +
CHAR(10) +
    'Fecha de corte: ' + CONVERT(VARCHAR, @FechaCorte, 103)
+ CHAR(13) + CHAR(10) +
    'Registros archivados: ' + CAST(@RegistrosMover AS
VARCHAR) + CHAR(13) + CHAR(10) +
    'Registros eliminados: ' + CAST(@RegistrosEliminar AS
VARCHAR) + CHAR(13) + CHAR(10) +
    'Total de registros afectados: ' + CAST(@RegistrosMover
+ @RegistrosEliminar AS VARCHAR);

-- Actualizar auditoría
UPDATE Auditoria.AuditoriaJobs
SET
    Estado = 'EXITOSO',
    Mensaje = @Mensaje,
    RegistrosAfectados = @RegistrosMover + @RegistrosEliminar,
    DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion, SYSDATETIME())
WHERE Id = @IdRegistro;

```

```

        PRINT @Mensaje;

    END TRY
    BEGIN CATCH
        -- Rollback si la transacción está activa
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION @TranName;

        SET @ErrorMsg = 'ERROR: ' + ERROR_MESSAGE() +
            ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) +
            ', Línea: ' + CAST(ERROR_LINE() AS VARCHAR) + ')';

        IF @IdRegistro IS NOT NULL
        BEGIN
            UPDATE Auditoria.AuditoriaJobs
            SET
                Estado = 'FALLIDO',
                Mensaje = @ErrorMsg,
                DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion,
SYSDATETIME())
            WHERE Id = @IdRegistro;
        END

        PRINT @ErrorMsg;
        THROW;
    END CATCH
END
GO

-- 4. CREAM JOB EN SQL SERVER AGENT
USE msdb;
GO

IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'ArchivarLogTemporal')
BEGIN
    EXEC sp_delete_job @job_name = 'ArchivarLogTemporal';
    PRINT 'Job existente eliminado.';
END
GO

-- Crear el job
EXEC sp_add_job
    @job_name = 'ArchivarLogTemporal',
    @enabled = 1,
    @description = 'Archivar y eliminar logs temporales > 180 días',
    @start_step_id = 1,
    @category_name = 'Database Maintenance',
    @owner_login_name = 'sa';
GO

-- Crear el step
EXEC sp_add_jobstep
    @job_name = 'ArchivarLogTemporal',
    @step_id = 1,
    @step_name = 'Archivar y eliminar logs',
    @command = 'EXEC GradosTitulos.Tramite.SP_ArchivarYEliminarLogTemporal;',
    @database_name = 'GradosTitulos',
    @subsystem = 'TSQL',
    @retry_attempts = 2,
    @retry_interval = 15;
GO

```

```

-- Crear schedule: Primer día de cada mes a las 01:00 AM
EXEC sp_add_jobschedule
    @job_name = 'ArchivarLogTemporal',
    @name = 'Mensual_Dia1_1AM',
    @freq_type = 16, -- Mensual
    @freq_interval = 1, -- Día 1 del mes
    @active_start_time = 010000; -- 01:00:00
GO

-- Asociar al servidor
EXEC sp_add_jobserver
    @job_name = 'ArchivarLogTemporal',
    @server_name = '(local)';
GO

PRINT 'Job "ArchivarLogTemporal" creado exitosamente.';
PRINT 'Programación: Primer día de cada mes a las 01:00 AM.';
GO

-- 5. VERIFICAR
USE GradosTitulos;
GO

SELECT
    j.name AS JobName,
    s.name AS ScheduleName,
    s.active_start_time AS HoraInicio,
    s.freq_interval AS DiaMes
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.sysjobschedules js ON j.job_id = js.job_id
JOIN msdb.dbo.sysschedules s ON js.schedule_id = s.schedule_id
WHERE j.name = 'ArchivarLogTemporal';
GO

```

Proyecto 4: Enunciado: Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a 17 durante el registro de trámites.
Codigo:

```

-- =====
-- PROYECTO 4: Alert para errores de severidad >= 17
-- Notificación: Operador de Grados y Títulos
-- =====

USE msdb;
GO

-- 1. CREAR OPERADOR PARA NOTIFICACIONES
IF NOT EXISTS (SELECT 1 FROM dbo.sysoperators WHERE name =
'OperadorGradosTitulos')
BEGIN
    EXEC sp_add_operator
        @name = 'OperadorGradosTitulos',
        @enabled = 1,
        @email_address = 'grados.titulos@upla.edu.pe',
        @pager_address = NULL,
        @weekday_pager_start_time = 080000,
        @weekday_pager_end_time = 180000,
        @saturday_pager_start_time = 080000,
        @saturday_pager_end_time = 130000,
        @sunday_pager_start_time = 0,
        @sunday_pager_end_time = 0,
        @pager_days = 62, -- Lunes a Sábado

```

```

        @netsend_address = NULL,
        @category_name = 'DBA';

    PRINT 'Operador "OperadorGradosTitulos" creado exitosamente.';
END
ELSE
BEGIN
    PRINT 'El operador "OperadorGradosTitulos" ya existe.';
END
GO

-- 2. CREAR ALERTA PARA ERRORES DE SEVERIDAD >= 17
IF EXISTS (SELECT 1 FROM dbo.sysalerts WHERE name = 'GradosTitulos_ErrorSevero')
BEGIN
    EXEC sp_delete_alert @name = 'GradosTitulos_ErrorSevero';
    PRINT 'Alerta existente eliminada.';
END
GO

EXEC sp_add_alert
    @name = 'GradosTitulos_ErrorSevero',
    @message_id = 0,
    @severity = 17, -- Severidad 17 (error de recursos)
    @enabled = 1,
    @delay_between_responses = 60, -- Minutos entre notificaciones
    @include_event_description_in = 1,
    @database_name = 'GradosTitulos',
    @job_id = NULL,
    @notification_message = 'Error de severidad >= 17 detectado en la base de
datos GradosTitulos.';
GO

-- 3. ASOCIAR LA ALERTA CON EL OPERADOR
EXEC sp_add_notification
    @alert_name = 'GradosTitulos_ErrorSevero',
    @operator_name = 'OperadorGradosTitulos',
    @notification_method = 1; -- Email
GO

PRINT 'Alerta "GradosTitulos_ErrorSevero" creada y asociada al operador.';
GO

-- 4. CREAR ALERTAS ADICIONALES PARA OTRAS SEVERIDADES (OPCIONAL)

-- Alerta para severidad 18 (error interno)
IF NOT EXISTS (SELECT 1 FROM dbo.sysalerts WHERE name =
'GradosTitulos_ErrorInterno')
BEGIN
    EXEC sp_add_alert
        @name = 'GradosTitulos_ErrorInterno',
        @severity = 18,
        @enabled = 1,
        @delay_between_responses = 30,
        @include_event_description_in = 1,
        @database_name = 'GradosTitulos',
        @notification_message = 'Error interno (severidad 18) en
GradosTitulos.';

    EXEC sp_add_notification
        @alert_name = 'GradosTitulos_ErrorInterno',
        @operator_name = 'OperadorGradosTitulos',
        @notification_method = 1;

```

```

        PRINT 'Alerta "GradosTitulos_ErrorInterno" creada.';
    END
GO

-- 5. CREAR ALERTA PARA ERRORES EN PROCEDIMIENTOS ESPECÍFICOS
IF EXISTS (SELECT 1 FROM dbo.sysalerts WHERE name =
'GradosTitulos_ErrorEnProcedimientos')
BEGIN
    EXEC sp_delete_alert @name = 'GradosTitulos_ErrorEnProcedimientos';
END
GO

-- Alerta para errores 1205 (deadlock) y 1222 (bloqueo)
EXEC sp_add_alert
    @name = 'GradosTitulos_ErrorEnProcedimientos',
    @message_id = 1205, -- Deadlock
    @severity = 0,
    @enabled = 1,
    @delay_between_responses = 15,
    @include_event_description_in = 1,
    @database_name = 'GradosTitulos',
    @notification_message = 'Se ha detectado un deadlock en la base de datos
GradosTitulos.';
GO

EXEC sp_add_notification
    @alert_name = 'GradosTitulos_ErrorEnProcedimientos',
    @operator_name = 'OperadorGradosTitulos',
    @notification_method = 1;
GO

PRINT 'Alerta "GradosTitulos_ErrorEnProcedimientos" creada para deadlocks.';
GO

-- 6. PROCEDIMIENTO PARA PROBAR LA ALERTA
-- Crear un procedimiento que genera un error de severidad 17
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE Administracion.SP_ProbarAlertaSeveridad17
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Mensaje NVARCHAR(2000) = 'Prueba de alerta: Error simulado de
severidad 17';

    -- RAISERROR con severidad 17
    RAISERROR(@Mensaje, 17, 1);

    -- También se puede usar THROW para errores más graves
    -- THROW 50017, @Mensaje, 1;
END
GO

-- 7. EJECUTAR PRUEBA DE LA ALERTA (DESCOMENTAR PARA EJECUTAR)
-- EXEC Administracion.SP_ProbarAlertaSeveridad17;
-- GO

-- 8. VERIFICAR ALERTAS CREADAS
USE msdb;

```

GO

```
SELECT
    a.name AS Alerta,
    a.severity AS Severidad,
    a.message_id AS MensajeID,
    a.enabled AS Habilitado,
    a.database_name AS BaseDatos,
    a.notification_message AS MensajeNotificacion,
    o.name AS Operador,
    o.email_address AS Email
FROM dbo.sysalerts a
LEFT JOIN dbo.sysnotifications n ON a.id = n.alert_id
LEFT JOIN dbo.sysoperators o ON n.operator_id = o.id
WHERE a.database_name = 'GradosTitulos'
    OR a.name LIKE '%GradosTitulos%'
ORDER BY a.severity DESC, a.name;
GO
```

```
-- 9. VERIFICAR HISTORIAL DE ALERTAS
-- SELECT * FROM dbo.sysalerts_history
```

Proyecto 5: Enunciado: Implementar un sistema automático que diariamente:
Codigo:

```
--
=====
-- PROYECTO 5: Sistema automático con Database Mail
-- Verificación diaria de trámites pendientes > 30 días
--
=====
```

USE GradosTitulos;
GO

```
-- 1. CREAR TABLAS PARA ESTADÍSTICAS DE EJECUCIÓN
IF OBJECT_ID('Auditoria.EstadisticasTramitesPendientes', 'U') IS NULL
BEGIN
    CREATE TABLE Auditoria.EstadisticasTramitesPendientes (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        FechaReporte DATE NOT NULL,
        TotalTramitesPendientes INT NOT NULL,
        PorTipo XML NULL,
        PorEstado XML NULL,
        TotalDiasPromedio INT NULL,
        MaxDiasPendiente INT NULL,
        FechaGeneracion DATETIME2 DEFAULT SYSDATETIME(),
        EmailEnviado BIT DEFAULT 0,
        FechaEmail DATETIME2 NULL
    );

    CREATE NONCLUSTERED INDEX IX_EstadisticasTramitesPendientes_Fecha
    ON Auditoria.EstadisticasTramitesPendientes (FechaReporte DESC);

    PRINT 'Tabla Auditoria.EstadisticasTramitesPendientes creada.';
```



```
END
GO
```

```
-- 2. PROCEDIMIENTO PARA VERIFICAR TRÁMITES PENDIENTES
CREATE OR ALTER PROCEDURE Auditoria.SP_VerificarTramitesPendientes
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @IdRegistro INT;
```

```
    DECLARE @FechaReporte DATE = CAST(GETDATE() AS DATE);
```

```
    DECLARE @TotalPendientes INT = 0;
```

```
    DECLARE @TotalDiasPromedio INT = 0;
```

```
    DECLARE @MaxDiasPendiente INT = 0;
```

```
    DECLARE @Mensaje NVARCHAR(MAX);
```

```
    DECLARE @Body NVARCHAR(MAX);
```

```
    DECLARE @DetallesXML XML;
```

```
    DECLARE @EmailEnviado BIT = 0;
```

```
    DECLARE @ErrorMsg NVARCHAR(4000);
```

```
BEGIN TRY
```

```
    -- Registrar inicio
```

```
    EXEC Auditoria.SP_RegistrarEjecucionJob
```

```
        @JobName = 'VerificacionTramitesPendientes',
```

```
        @StepName = 'Verificar trámites pendientes > 30 días',
```

```
        @Estado = 'EN_EJECUCION',
```

```
        @IdRegistro = @IdRegistro OUTPUT;
```

```
    -- Crear tabla temporal para resultados
```

```
    CREATE TABLE #TramitesPendientes (
```

```
        IdTramite INT,
```

```
        IdMatricula INT,
```

```
        IdTipoTramite TINYINT,
```

```
        TipoTramite VARCHAR(60),
```

```
        IdEstadoTramite TINYINT,
```

```
        Estado VARCHAR(40),
```

```
        FechaInicio DATE,
```

```
        DiasPendientes INT,
```

```
        Observaciones VARCHAR(500)
```

```
    );
```

```
    -- Obtener trámites pendientes > 30 días (estados no terminales)
```

```
    INSERT INTO #TramitesPendientes
```

```
    SELECT
```

```
        t.IdTramite,
```

```
        t.IdMatricula,
```

```
        t.IdTipoTramite,
```

```
        tt.Descripcion AS TipoTramite,
```

```
        t.IdEstadoTramite,
```

```
        et.Descripcion AS Estado,
```

```

        CAST(t.FechaInicio AS DATE) AS FechaInicio,
        DATEDIFF(DAY, t.FechaInicio, GETDATE()) AS DiasPendientes,
        t.Observaciones
FROM Tramite.Tramite t
JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite
JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite = et.IdEstadoTramite
WHERE et.EsTerminal = 0 -- Estados no terminales (INICIADO,
EN_PROCESO, etc.)
    AND DATEDIFF(DAY, t.FechaInicio, GETDATE()) > 30;

SET @TotalPendientes = @@ROWCOUNT;

-- Calcular estadísticas
SELECT
    @TotalDiasPromedio = AVG(DiasPendientes),
    @MaxDiasPendiente = MAX(DiasPendientes)
FROM #TramitesPendientes;

-- Generar XML con detalles
SET @DetallesXML = (
    SELECT
        t.IdTramite,
        t.TipoTramite,
        t.Estado,
        t.FechaInicio,
        t.DiasPendientes,
        t.Observaciones
    FROM #TramitesPendientes t
    FOR XML AUTO, ROOT('TramitesPendientes')
);

-- Guardar estadísticas
INSERT INTO Auditoria.EstadisticasTramitesPendientes (
    FechaReporte,
    TotalTramitesPendientes,
    PorTipo,
    PorEstado,
    TotalDiasPromedio,
    MaxDiasPendiente,
    FechaGeneracion
)
SELECT
    @FechaReporte,
    @TotalPendientes,
    (SELECT TipoTramite, COUNT(*) AS Cantidad
    FROM #TramitesPendientes
    GROUP BY TipoTramite
    FOR XML AUTO, ROOT('PorTipo')),
    (SELECT Estado, COUNT(*) AS Cantidad
    FROM #TramitesPendientes

```

```

GROUP BY Estado
FOR XML AUTO, ROOT('PorEstado')),
@TotalDiasPromedio,
@MaxDiasPendiente,
SYSDATETIME();

-- Construir mensaje para el log
SET @Mensaje = 'VERIFICACIÓN DE TRÁMITES PENDIENTES' +
CHAR(13) + CHAR(10) +
'Fecha del reporte: ' + CONVERT(VARCHAR, @FechaReporte, 103)
+ CHAR(13) + CHAR(10) +
'Total de trámites pendientes > 30 días: ' + CAST(@TotalPendientes
AS VARCHAR) + CHAR(13) + CHAR(10) +
'Promedio de días pendientes: ' + CAST(@TotalDiasPromedio AS
VARCHAR) + CHAR(13) + CHAR(10) +
'Máximo de días pendientes: ' + CAST(@MaxDiasPendiente AS
VARCHAR);

-- Actualizar auditoría del job
UPDATE Auditoria.AuditoriaJobs
SET
Estado = 'EXITOSO',
Mensaje = @Mensaje,
RegistrosAfectados = @TotalPendientes,
DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion,
SYSDATETIME())
WHERE Id = @IdRegistro;

-- 3. ENVIAR CORREO ELECTRÓNICO
-- Construir cuerpo del correo
SET @Body = '<html><head><style>
body { font-family: Arial, sans-serif; }
h1 { color: #2c3e50; border-bottom: 2px solid #3498db; }
.total { font-size: 24px; font-weight: bold; color: #e74c3c; }
table { border-collapse: collapse; width: 100%; margin-top: 20px; }
th { background-color: #3498db; color: white; padding: 10px; text-align: left; }
td { border: 1px solid #ddd; padding: 8px; }
tr:nth-child(even) { background-color: #f2f2f2; }
.warning { color: #e74c3c; font-weight: bold; }
</style></head><body>';

SET @Body = @Body + '<h1>REPORTE DE TRÁMITES
PENDIENTES</h1>';

SET @Body = @Body + '<p>Fecha del reporte: <strong>' +
CONVERT(VARCHAR, @FechaReporte, 103) + '</strong></p>';

SET @Body = @Body + '<p>Total de trámites pendientes > 30 días: <span
class="total">' + CAST(@TotalPendientes AS VARCHAR) + '</span></p>';

SET @Body = @Body + '<p>Promedio de días pendientes: ' +
CAST(@TotalDiasPromedio AS VARCHAR) + '</p>';

```

```

SET @Body = @Body + '<p>Máximo de días pendientes: ' +
CAST(@MaxDiasPendiente AS VARCHAR) + '</p>';

-- Agregar tabla de detalles
IF @TotalPendientes > 0
BEGIN
    SET @Body = @Body + '<h2>Detalle de trámites pendientes</h2>';
    SET @Body = @Body +
'<table><tr><th>ID</th><th>Tipo</th><th>Estado</th><th>Fecha
Inicio</th><th>Días Pendientes</th></tr>';

    SELECT @Body = @Body +
        '<tr>' +
        '<td>' + CAST(IdTramite AS VARCHAR) + '</td>' +
        '<td>' + TipoTramite + '</td>' +
        '<td>' + Estado + '</td>' +
        '<td>' + CONVERT(VARCHAR, FechaInicio, 103) + '</td>' +
        '<td class="warning">' + CAST(DiasPendientes AS VARCHAR) + '</td>'
+
        '</tr>'
    FROM #TramitesPendientes
    ORDER BY DiasPendientes DESC;

    SET @Body = @Body + '</table>';
END

-- Agregar gráfico de distribución (simplificado)
SET @Body = @Body + '<h2>Distribución por Tipo de Trámite</h2>';
SET @Body = @Body + '<table><tr><th>Tipo</th><th>Cantidad</th></tr>';

SELECT @Body = @Body +
    '<tr><td>' + TipoTramite + '</td><td>' + CAST(Cantidad AS VARCHAR) +
'</td></tr>'
FROM (
    SELECT TipoTramite, COUNT(*) AS Cantidad
    FROM #TramitesPendientes
    GROUP BY TipoTramite
) AS t
ORDER BY Cantidad DESC;

SET @Body = @Body + '</table>';

SET @Body = @Body + '<p style="margin-top: 30px; font-size: 12px; color:
#7f8c8d;">';
SET @Body = @Body + 'Este es un reporte automático generado por SQL
Server Agent.';
SET @Body = @Body + '<br>Por favor, no responda a este correo.';
SET @Body = @Body + '</p></body></html>';

-- Enviar correo solo si hay trámites pendientes

```

```

IF @TotalPendientes > 0
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'GradosTitulos_Profile',
        @recipients = 'grados.titulos@upla.edu.pe; administracion@upla.edu.pe',
        @copy_recipients = 'dba@upla.edu.pe',
        @subject = 'REPORTE: Trámites pendientes > 30 días - ' +
CONVERT(VARCHAR, GETDATE(), 103),
        @body = @Body,
        @body_format = 'HTML',
        @importance = 'HIGH';

    SET @EmailEnviado = 1;
END

-- Actualizar estado de email
UPDATE Auditoria.EstadisticasTramitesPendientes
SET
    EmailEnviado = @EmailEnviado,
    FechaEmail = CASE WHEN @EmailEnviado = 1 THEN SYSDATETIME()
ELSE NULL END
WHERE FechaReporte = @FechaReporte
AND Id = SCOPE_IDENTITY();

-- Limpiar tabla temporal
DROP TABLE #TramitesPendientes;

PRINT @Mensaje;
IF @EmailEnviado = 1
    PRINT 'Correo enviado exitosamente.';
ELSE
    PRINT 'No se envió correo (no hay trámites pendientes).';

END TRY
BEGIN CATCH
    SET @ErrorMsg = 'ERROR: ' + ERROR_MESSAGE() +
        ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) + ')';

    IF @IdRegistro IS NOT NULL
    BEGIN
        UPDATE Auditoria.AuditoriaJobs
        SET
            Estado = 'FALLIDO',
            Mensaje = @ErrorMsg,
            DuracionMs = DATEDIFF(MILLISECOND, FechaEjecucion,
SYSDATETIME())
        WHERE Id = @IdRegistro;
    END

    -- Intentar enviar correo de error

```

```

BEGIN TRY
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'GradosTitulos_Profile',
        @recipients = 'dba@upla.edu.pe',
        @subject = 'ERROR: Verificación de trámites pendientes',
        @body = 'Error en la verificación de trámites pendientes:' + CHAR(13) +
CHAR(10) +
        @ErrorMsg,
        @body_format = 'TEXT';
END TRY
BEGIN CATCH
    -- Silenciar error de email
END CATCH

PRINT @ErrorMsg;
THROW;
END CATCH
END
GO

```

-- 4. CREAR JOB EN SQL SERVER AGENT

```
USE msdb;
```

```
GO
```

```
IF EXISTS (SELECT 1 FROM sysjobs WHERE name =
'VerificacionTramitesPendientes')
```

```
BEGIN
```

```
    EXEC sp_delete_job @job_name = 'VerificacionTramitesPendientes';
```

```
    PRINT 'Job existente eliminado.';
```

```
END
```

```
GO
```

-- Crear el job

```
EXEC sp_add_job
```

```
    @job_name = 'VerificacionTramitesPendientes',
```

```
    @enabled = 1,
```

```
    @description = 'Verificación diaria de trámites pendientes > 30 días con envío de
correo',
```

```
    @start_step_id = 1,
```

```
    @category_name = 'Database Maintenance',
```

```
    @owner_login_name = 'sa';
```

```
GO
```

-- Crear el step

```
EXEC sp_add_jobstep
```

```
    @job_name = 'VerificacionTramitesPendientes',
```

```
    @step_id = 1,
```

```
    @step_name = 'Verificar y reportar trámites pendientes',
```

```
    @command = 'EXEC GradosTitulos.Auditoria.SP_VerificarTramitesPendientes;',
```

```
    @database_name = 'GradosTitulos',
```

```

    @subsystem = 'TSQL',
    @retry_attempts = 3,
    @retry_interval = 10;
GO

-- Crear schedule: Diario a las 07:00 AM
EXEC sp_add_jobschedule
    @job_name = 'VerificacionTramitesPendientes',
    @name = 'Diario_7AM',
    @freq_type = 4, -- Diario
    @freq_interval = 1, -- Todos los días
    @active_start_time = 070000; -- 07:00:00
GO

-- Asociar al servidor
EXEC sp_add_jobserver
    @job_name = 'VerificacionTramitesPendientes',
    @server_name = '(local)';
GO

PRINT 'Job "VerificacionTramitesPendientes" creado exitosamente.';
PRINT 'Programación: Todos los días a las 07:00 AM.';
PRINT 'Nota: Asegurarse de que Database Mail esté configurado con el perfil "GradosTitulos_Profile"';
GO

-- 5. VERIFICAR CONFIGURACIÓN DE DATABASE MAIL
-- Verificar perfiles de Database Mail
USE msdb;
GO

SELECT
    profile_id,
    name AS ProfileName,
    description AS Descripcion
FROM dbo.sysmail_profile
WHERE name LIKE '%Grados%' OR name LIKE '%Titulos%';
GO

-- Si no existe el perfil, crearlo (ejecutar después de configurar Database Mail)
IF NOT EXISTS (SELECT 1 FROM dbo.sysmail_profile WHERE name = 'GradosTitulos_Profile')
BEGIN
    PRINT 'ADVERTENCIA: No se encontró el perfil "GradosTitulos_Profile"';
    PRINT 'Configurar Database Mail y crear el perfil.';
END
ELSE
BEGIN
    PRINT 'Perfil "GradosTitulos_Profile" encontrado.';
END

```

GO

-- 6. VERIFICAR ESTADÍSTICAS GUARDADAS

USE GradosTitulos;

GO

SELECT TOP 5

 FechaReporte,
 TotalTramitesPendientes,
 TotalDiasPromedio,
 MaxDiasPendiente,
 EmailEnviado,
 FechaEmail

FROM Auditoria.EstadisticasTramitesPendientes

ORDER BY FechaReporte DESC;

GO

-- 7. EJECUTAR EL JOB MANUALMENTE PARA PRUEBA

-- EXEC msdb.dbo.sp_start_job @job_name = 'VerificacionTramitesPendientes';

-- GO

-- 8. CONSULTAR HISTORIAL COMPLETO DE EJECUCIÓN

SELECT

 JobName,
 FechaEjecucion,
 Estado,
 RegistrosAfectados,
 DuracionMs,
 LEFT(Mensaje, 200) AS Mensaje

FROM Auditoria.AuditoriaJobs

WHERE JobName IN ('VerificacionTramitesPendientes', 'ReporteDiarioTramites')

ORDER BY FechaEjecucion DESC;

GO

TEMA 2. Uso de Database Maintenance Plans

Proyecto 1: Enunciado Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos **GradosTitulos**.

Codigo:

```
-- =====
-- PROYECTO 1: Backup completo diario con compresión y verificación
-- Ejecución: Diario a las 23:00 horas
-- =====

USE master;
GO

-- 1. CREAR CARPETA PARA BACKUPS (SI NO EXISTE)
-- NOTA: Ejecutar desde Windows o crear la carpeta manualmente
-- EXEC xp_create_subdir 'D:\Backups\GradosTitulos\';
-- GO

-- 2. CREAR TABLA DE AUDITORÍA PARA BACKUPS
USE GradosTitulos;
GO

IF OBJECT_ID('Auditoria.HistorialBackups', 'U') IS NULL
BEGIN
    CREATE TABLE Auditoria.HistorialBackups (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        FechaBackup DATETIME2 DEFAULT SYSDATETIME(),
        TipoBackup VARCHAR(20) NOT NULL, -- COMPLETO, DIFERENCIAL, LOG
        NombreArchivo NVARCHAR(500) NOT NULL,
        TamanioMB DECIMAL(18,2) NULL,
        DuracionSegundos INT NULL,
        Compresion BIT DEFAULT 1,
        Verificado BIT DEFAULT 0,
        Estado VARCHAR(20) NOT NULL, -- EXITOSO, FALLIDO
        Mensaje NVARCHAR(MAX) NULL
    );

    CREATE NONCLUSTERED INDEX IX_HistorialBackups_Fecha
    ON Auditoria.HistorialBackups (FechaBackup DESC);

    PRINT 'Tabla Auditoria.HistorialBackups creada.';
END
GO

-- 3. PROCEDIMIENTO PARA BACKUP COMPLETO
CREATE OR ALTER PROCEDURE Administracion.SP_BackupCompletoGradosTitulos
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @IdRegistro INT;
    DECLARE @BackupPath NVARCHAR(500);
    DECLARE @BackupName NVARCHAR(500);
    DECLARE @FechaStr VARCHAR(20);
    DECLARE @ErrorMsg NVARCHAR(4000);
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
    DECLARE @TamanioMB DECIMAL(18,2);
    DECLARE @DuracionSeg INT;
```

```

BEGIN TRY
-- Registrar inicio
INSERT INTO Auditoria.HistorialBackups (
    TipoBackup,
    NombreArchivo,
    Estado,
    Compresion,
    Verificado
)
VALUES (
    'COMPLETO',
    'Iniciando backup...',
    'EN_EJECUCION',
    1,
    0
);

SET @IdRegistro = SCOPE_IDENTITY();

-- Generar nombre del archivo
SET @FechaStr = CONVERT(VARCHAR, GETDATE(), 112) + '_' +
    REPLACE(CONVERT(VARCHAR, GETDATE(), 108), ':', '');
SET @BackupName = 'GradosTitulos_Backup_' + @FechaStr;
SET @BackupPath = 'D:\Backups\GradosTitulos\' + @BackupName + '.bak';

-- Ejecutar backup con compresión
BACKUP DATABASE [GradosTitulos]
TO DISK = @BackupPath
WITH
    COMPRESSION,
    NAME = @BackupName,
    DESCRIPTION = 'Backup completo de GradosTitulos - ' +
CONVERT(VARCHAR, GETDATE(), 120),
    STATS = 10,
    CHECKSUM,
    INIT,
    SKIP,
    NOREWIND,
    NOUNLOAD,
    FORMAT;

-- Verificar el backup
RESTORE VERIFYONLY
FROM DISK = @BackupPath
WITH
    CHECKSUM,
    CONTINUE_AFTER_ERROR;

-- Obtener tamaño del archivo
DECLARE @FileSize BIGINT;
EXEC xp_fileexist @BackupPath, @FileSize OUTPUT;
SET @TamanoMB = @FileSize / 1048576.0;

-- Calcular duración
SET @DuracionSeg = DATEDIFF(SECOND, @StartTime, SYSDATETIME());

-- Actualizar registro
UPDATE Auditoria.HistorialBackups
SET
    NombreArchivo = @BackupPath,
    TamanoMB = @TamanoMB,
    DuracionSegundos = @DuracionSeg,

```

```

        Verificado = 1,
        Estado = 'EXITOSO',
        Mensaje = 'Backup completado exitosamente. Tamaño: ' +
                  CAST(@TamanioMB AS VARCHAR) + ' MB, Duración: ' +
                  CAST(@DuracionSeg AS VARCHAR) + ' segundos'
WHERE Id = @IdRegistro;

-- Guardar información del backup en msdb (para restaurar fácilmente)
-- Los backups ya quedan registrados en msdb automáticamente

PRINT 'Backup completado: ' + @BackupPath;
PRINT 'Tamaño: ' + CAST(@TamanioMB AS VARCHAR) + ' MB';
PRINT 'Duración: ' + CAST(@DuracionSeg AS VARCHAR) + ' segundos';

END TRY
BEGIN CATCH
    SET @ErrorMsg = 'ERROR en backup: ' + ERROR_MESSAGE() +
                    ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) + ')';

    IF @IdRegistro IS NOT NULL
    BEGIN
        UPDATE Auditoria.HistorialBackups
        SET
            Estado = 'FALLIDO',
            Mensaje = @ErrorMsg,
            DuracionSegundos = DATEDIFF(SECOND, @StartTime, SYSDATETIME())
        WHERE Id = @IdRegistro;
    END

    PRINT @ErrorMsg;
    THROW;
END CATCH

END
GO

-- 4. CREAR JOB EN SQL SERVER AGENT
USE msdb;
GO

-- Eliminar job si existe
IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'BackupCompletoGradosTitulos')
BEGIN
    EXEC sp_delete_job @job_name = 'BackupCompletoGradosTitulos';
    PRINT 'Job existente eliminado.';
END
GO

-- Crear el job
EXEC sp_add_job
    @job_name = 'BackupCompletoGradosTitulos',
    @enabled = 1,
    @description = 'Backup completo diario de la base de datos GradosTitulos
con compresión y verificación',
    @start_step_id = 1,
    @category_name = 'Database Maintenance',
    @owner_login_name = 'sa';
GO

-- Crear el step
EXEC sp_add_jobstep
    @job_name = 'BackupCompletoGradosTitulos',
    @step_id = 1,

```

```

        @step_name = 'Ejecutar backup completo',
        @command = 'EXEC
GradosTitulos.Administracion.SP_BackupCompletoGradosTitulos;',
        @database_name = 'GradosTitulos',
        @subsystem = 'TSQL',
        @retry_attempts = 3,
        @retry_interval = 5;
GO

-- Crear schedule: Diario a las 23:00 horas
EXEC sp_add_jobschedule
    @job_name = 'BackupCompletoGradosTitulos',
    @name = 'Diario_11PM',
    @freq_type = 4, -- Diario
    @freq_interval = 1, -- Todos los días
    @active_start_time = 230000; -- 23:00:00
GO

-- Asociar al servidor
EXEC sp_add_jobserver
    @job_name = 'BackupCompletoGradosTitulos',
    @server_name = '(local)';
GO

PRINT 'Job "BackupCompletoGradosTitulos" creado exitosamente.';
PRINT 'Programación: Todos los días a las 23:00 horas.';
GO

-- 5. VERIFICAR BACKUPS REALIZADOS
USE GradosTitulos;
GO

SELECT TOP 5
    FechaBackup,
    TipoBackup,
    TamanoMB,
    DuracionSegundos,
    Estado,
    LEFT(NombreArchivo, 80) AS Archivo
FROM Auditoria.HistorialBackups
ORDER BY FechaBackup DESC;

GO

```

Proyecto 2: Enunciado Crear un plan semanal para reorganizar índices de las tablas:

Codigo:

```

-- =====
-- PROYECTO 2: Reorganización semanal de índices específicos
-- Tablas: Persona, Tramite, TrabajoInvestigacion, Sustentacion
-- Ejecución: Domingos a las 03:00 AM
-- =====

USE GradosTitulos;
GO

-- 1. TABLA DE AUDITORÍA PARA MANTENIMIENTO DE ÍNDICES
IF OBJECT_ID('Auditoria.MantenimientoIndices', 'U') IS NULL
BEGIN
    CREATE TABLE Auditoria.MantenimientoIndices (

```

```

        Id INT IDENTITY(1,1) PRIMARY KEY,
        FechaEjecucion DATETIME2 DEFAULT SYSDATETIME(),
        Tabla NVARCHAR(128) NOT NULL,
        Indice NVARCHAR(128) NOT NULL,
        TipoOperacion VARCHAR(20) NOT NULL, -- REORGANIZE, REBUILD
        FragmentacionAntes DECIMAL(5,2) NOT NULL,
        FragmentacionDespues DECIMAL(5,2) NULL,
        DuracionMs INT NULL,
        Estado VARCHAR(20) NOT NULL,
        Mensaje NVARCHAR(MAX) NULL
    );

    PRINT 'Tabla Auditoria.MantenimientoIndices creada.';
END
GO

-- 2. PROCEDIMIENTO PARA REORGANIZAR ÍNDICES
CREATE OR ALTER PROCEDURE Administracion.SP_ReorganizarIndicesSemanales
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Tabla NVARCHAR(128);
    DECLARE @Indice NVARCHAR(128);
    DECLARE @SchemaName NVARCHAR(128);
    DECLARE @Fragmentacion DECIMAL(5,2);
    DECLARE @Command NVARCHAR(1000);
    DECLARE @ErrorMsg NVARCHAR(4000);
    DECLARE @IdRegistro INT;
    DECLARE @StartTime DATETIME2;

    -- Tabla temporal para almacenar índices a procesar
    CREATE TABLE #IndicesAProcesar (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        SchemaName NVARCHAR(128),
        TableName NVARCHAR(128),
        IndexName NVARCHAR(128),
        Fragmentation DECIMAL(5,2)
    );

    -- Obtener índices de las tablas especificadas con fragmentación > 5%
    INSERT INTO #IndicesAProcesar (SchemaName, TableName, IndexName,
    Fragmentation)
    SELECT
        OBJECT_SCHEMA_NAME(ips.object_id) AS SchemaName,
        OBJECT_NAME(ips.object_id) AS TableName,
        i.name AS IndexName,
        ips.avg_fragmentation_in_percent AS Fragmentation
    FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL,
    NULL, 'LIMITED') ips
    JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
    i.index_id
    WHERE OBJECT_NAME(ips.object_id) IN ('Persona', 'Tramite',
    'TrabajoInvestigacion', 'Sustentacion')
    AND i.type > 0 -- No HEAP
    AND ips.avg_fragmentation_in_percent > 5.0
    AND ips.page_count > 50
    ORDER BY ips.avg_fragmentation_in_percent DESC;

    -- Declarar cursor
    DECLARE idx_cursor CURSOR FOR
        SELECT SchemaName, TableName, IndexName, Fragmentation

```

```

        FROM #IndicesAProcesar
        ORDER BY Fragmentation DESC;

    OPEN idx_cursor;
    FETCH NEXT FROM idx_cursor INTO @SchemaName, @Tabla, @Indice,
    @Fragmentacion;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @StartTime = SYSDATETIME();

        BEGIN TRY
            -- Registrar inicio
            INSERT INTO Auditoria.MantenimientoIndices (
                Tabla,
                Indice,
                TipoOperacion,
                FragmentacionAntes,
                Estado
            )
            VALUES (
                @SchemaName + '.' + @Tabla,
                @Indice,
                'REORGANIZE',
                @Fragmentacion,
                'EN_EJECUCION'
            );

            SET @IdRegistro = SCOPE_IDENTITY();

            -- Ejecutar REORGANIZE
            SET @Command = 'ALTER INDEX [' + @Indice + '] ON [' + @SchemaName +
            '].[' + @Tabla + '] REORGANIZE;';

            PRINT 'Reorganizando: ' + @Command;
            EXEC sp_executesql @Command;

            -- Obtener fragmentación después
            DECLARE @FragDespues DECIMAL(5,2);
            SELECT TOP 1 @FragDespues = avg_fragmentation_in_percent
            FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'),
                OBJECT_ID(@SchemaName + '.' + @Tabla),
                NULL, NULL, 'LIMITED')
            WHERE index_id = (SELECT index_id FROM sys.indexes
                WHERE name = @Indice AND object_id =
            OBJECT_ID(@SchemaName + '.' + @Tabla));

            -- Actualizar registro
            UPDATE Auditoria.MantenimientoIndices
            SET
                FragmentacionDespues = @FragDespues,
                DuracionMs = DATEDIFF(MILLISECOND, @StartTime, SYSDATETIME()),
                Estado = 'EXITOSO',
                Mensaje = 'Fragmentación: ' + CAST(@Fragmentacion AS VARCHAR) +
                '% → ' +
                CAST(@FragDespues AS VARCHAR) + '%'
            WHERE Id = @IdRegistro;

        END TRY
        BEGIN CATCH
            SET @ErrorMsg = 'Error en índice ' + @Indice + ': ' +
            ERROR_MESSAGE();
        
```

```

        IF @IdRegistro IS NOT NULL
        BEGIN
            UPDATE Auditoria.MantenimientoIndices
            SET
                Estado = 'FALLIDO',
                Mensaje = @ErrorMsg,
                DuracionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME())
            WHERE Id = @IdRegistro;
        END

        PRINT @ErrorMsg;
    END CATCH

    FETCH NEXT FROM idx_cursor INTO @SchemaName, @Tabla, @Indice,
@Fragmentacion;
    END

    CLOSE idx_cursor;
    DEALLOCATE idx_cursor;
    DROP TABLE #IndicesAProcesar;

    -- Resumen
    DECLARE @Exitosos INT, @Fallidos INT;
    SELECT @Exitosos = COUNT(*) FROM Auditoria.MantenimientoIndices
    WHERE FechaEjecucion >= DATEADD(HOUR, -1, SYSDATETIME()) AND Estado =
'EXITOSO';
    SELECT @Fallidos = COUNT(*) FROM Auditoria.MantenimientoIndices
    WHERE FechaEjecucion >= DATEADD(HOUR, -1, SYSDATETIME()) AND Estado =
'FALLIDO';

    PRINT 'Resumen de reorganización: ' + CAST(@Exitosos AS VARCHAR) + '
exitosos, ' +
        CAST(@Fallidos AS VARCHAR) + ' fallidos.';
    END
GO

-- 3. CREAR JOB EN SQL SERVER AGENT
USE msdb;
GO

IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'ReorganizacionIndicesSemanales')
BEGIN
    EXEC sp_delete_job @job_name = 'ReorganizacionIndicesSemanales';
    PRINT 'Job existente eliminado.';
END
GO

EXEC sp_add_job
    @job_name = 'ReorganizacionIndicesSemanales',
    @enabled = 1,
    @description = 'Reorganización semanal de índices: Persona, Tramite,
TrabajoInvestigacion, Sustentacion',
    @start_step_id = 1,
    @category_name = 'Database Maintenance',
    @owner_login_name = 'sa';
GO

EXEC sp_add_jobstep
    @job_name = 'ReorganizacionIndicesSemanales',
    @step_id = 1,

```

```

        @step_name = 'Reorganizar índices',
        @command = 'EXEC
GradosTitulos.Administracion.SP_ReorganizarIndicesSemanales;',
        @database_name = 'GradosTitulos',
        @subsystem = 'TSQL',
        @retry_attempts = 2,
        @retry_interval = 10;
GO

-- Programación: Domingos a las 03:00 AM
EXEC sp_add_jobschedule
    @job_name = 'ReorganizacionIndicesSemanales',
    @name = 'Semanal_Domingo_3AM',
    @freq_type = 8, -- Semanal
    @freq_interval = 1, -- Domingo
    @active_start_time = 030000;
GO

EXEC sp_add_jobserver
    @job_name = 'ReorganizacionIndicesSemanales',
    @server_name = '(local)';
GO

PRINT 'Job "ReorganizacionIndicesSemanales" creado.';

GO

```

Proyecto 3: Enunciado Diseñar un plan que:

- xreorganice índices menores al 30 %
- reconstruya índices mayores al 30 %
- actualice estadísticas.

Codigo:

```

-- =====
-- PROYECTO 3: Mantenimiento inteligente de índices
-- Reorganizar (< 30%), Rebuild (> 30%), Actualizar estadísticas
-- =====

USE GradosTitulos;
GO

-- 1. PROCEDIMIENTO DE MANTENIMIENTO INTELIGENTE
CREATE OR ALTER PROCEDURE Administracion.SP_MantenimientoIndicesInteligente
    @ReorganizeThreshold DECIMAL(5,2) = 5.0,
    @RebuildThreshold DECIMAL(5,2) = 30.0,
    @MinPages INT = 100,
    @UpdateStatistics BIT = 1
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SchemaName NVARCHAR(128);
    DECLARE @TableName NVARCHAR(128);
    DECLARE @IndexName NVARCHAR(128);
    DECLARE @IndexID INT;
    DECLARE @Fragmentation DECIMAL(5,2);
    DECLARE @PageCount INT;
    DECLARE @Command NVARCHAR(2000);
    DECLARE @OperationType VARCHAR(20);

```



```

DECLARE @StartTime DATETIME2;
DECLARE @IdRegistro INT;
DECLARE @ErrorMsg NVARCHAR(4000);
DECLARE @TotalReorganize INT = 0;
DECLARE @TotalRebuild INT = 0;
DECLARE @TotalStats INT = 0;

-- Tabla temporal para índices
CREATE TABLE #IndicesMantenimiento (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    SchemaName NVARCHAR(128),
    TableName NVARCHAR(128),
    IndexName NVARCHAR(128),
    IndexID INT,
    Fragmentation DECIMAL(5,2),
    PageCount INT,
    OperationType VARCHAR(20) -- REORGANIZE, REBUILD, NONE
);

-- Obtener todos los índices con fragmentación
INSERT INTO #IndicesMantenimiento (
    SchemaName, TableName, IndexName, IndexID, Fragmentation, PageCount,
OperationType
)
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) AS SchemaName,
    OBJECT_NAME(ips.object_id) AS TableName,
    i.name AS IndexName,
    ips.index_id AS IndexID,
    ips.avg_fragmentation_in_percent AS Fragmentation,
    ips.page_count AS PageCount,
    CASE
        WHEN ips.avg_fragmentation_in_percent >= @RebuildThreshold THEN
'REBUILD'
        WHEN ips.avg_fragmentation_in_percent >= @ReorganizeThreshold THEN
'REORGANIZE'
        ELSE 'NONE'
    END AS OperationType
FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL,
NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
i.index_id
WHERE i.type > 0 -- No HEAP
    AND ips.page_count > @MinPages
    AND ips.avg_fragmentation_in_percent >= @ReorganizeThreshold
ORDER BY ips.avg_fragmentation_in_percent DESC;

-- Declarar cursor
DECLARE idx_cursor CURSOR FOR
    SELECT SchemaName, TableName, IndexName, IndexID, Fragmentation,
PageCount, OperationType
FROM #IndicesMantenimiento
WHERE OperationType IN ('REORGANIZE', 'REBUILD')
ORDER BY OperationType DESC, Fragmentation DESC;

OPEN idx_cursor;
FETCH NEXT FROM idx_cursor INTO @SchemaName, @TableName, @IndexName,
@IndexID,
                                @Fragmentation, @PageCount, @OperationType;

WHILE @@FETCH_STATUS = 0
BEGIN

```

```

SET @StartTime = SYSDATETIME();

BEGIN TRY
    -- Registrar inicio
    INSERT INTO Auditoria.MantenimientoIndices (
        Tabla,
        Indice,
        TipoOperacion,
        FragmentacionAntes,
        Estado
    )
    VALUES (
        @SchemaName + '.' + @TableName,
        @IndexName,
        @OperationType,
        @Fragmentation,
        'EN_EJECUCION'
    );

    SET @IdRegistro = SCOPE_IDENTITY();

    -- Construir comando
    IF @OperationType = 'REORGANIZE'
    BEGIN
        SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +
@SchemaName + '].[' + @TableName + '] REORGANIZE;';
        SET @TotalReorganize = @TotalReorganize + 1;
    END
    ELSE IF @OperationType = 'REBUILD'
    BEGIN
        -- Usar ONLINE = ON si es Enterprise, OFF si no
        SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +
@SchemaName + '].[' + @TableName + '] REBUILD WITH (ONLINE = OFF,
SORT_IN_TEMPDB = ON);';
        SET @TotalRebuild = @TotalRebuild + 1;
    END

    PRINT 'Ejecutando: ' + @Command;
    EXEC sp_executesql @Command;

    -- Actualizar estadísticas si está habilitado
    IF @UpdateStatistics = 1
    BEGIN
        SET @Command = 'UPDATE STATISTICS [' + @SchemaName + '].[' +
@TableName + '] [' + @IndexName + '] WITH FULLSCAN;';
        EXEC sp_executesql @Command;
        SET @TotalStats = @TotalStats + 1;
    END

    -- Obtener fragmentación después
    DECLARE @FragDespues DECIMAL(5,2);
    SELECT TOP 1 @FragDespues = avg_fragmentation_in_percent
    FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'),
        OBJECT_ID(@SchemaName + '.' + @TableName),
        @IndexID, NULL, 'LIMITED');

    -- Actualizar registro
    UPDATE Auditoria.MantenimientoIndices
    SET
        FragmentacionDespues = @FragDespues,
        DuracionMs = DATEDIFF(MILLISECOND, @StartTime, SYSDATETIME()),
        Estado = 'EXITOSO',

```

```

        Mensaje = 'Fragmentación: ' + CAST(@Fragmentation AS VARCHAR) +
'% → ' +
        CAST(ISNULL(@FragDespues, 0) AS VARCHAR) + '% |
Páginas: ' + CAST(@PageCount AS VARCHAR)
        WHERE Id = @IdRegistro;

    END TRY
    BEGIN CATCH
        SET @ErrorMsg = 'Error en ' + @OperationType + ' de ' + @IndexName
+ ': ' + ERROR_MESSAGE();

        IF @IdRegistro IS NOT NULL
        BEGIN
            UPDATE Auditoria.MantenimientoIndices
            SET
                Estado = 'FALLIDO',
                Mensaje = @ErrorMsg,
                DuracionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME())
            WHERE Id = @IdRegistro;
        END

        PRINT @ErrorMsg;
    END CATCH

    FETCH NEXT FROM idx_cursor INTO @SchemaName, @TableName, @IndexName,
@IndexID,
                                @Fragmentation, @PageCount,
@OperationType;
    END

    CLOSE idx_cursor;
    DEALLOCATE idx_cursor;
    DROP TABLE #IndicesMantenimiento;

    -- Resumen final
    PRINT '=====';
    PRINT 'RESUMEN DE MANTENIMIENTO INTELIGENTE';
    PRINT '=====';
    PRINT 'Índices REORGANIZADOS: ' + CAST(@TotalReorganize AS VARCHAR);
    PRINT 'Índices REBUILT: ' + CAST(@TotalRebuild AS VARCHAR);
    PRINT 'Estadísticas actualizadas: ' + CAST(@TotalStats AS VARCHAR);
    PRINT 'Total procesados: ' + CAST(@TotalReorganize + @TotalRebuild AS
VARCHAR);
    PRINT '=====';
END
GO

-- 2. CREAR JOB
USE msdb;
GO

IF EXISTS (SELECT 1 FROM sysjobs WHERE name = 'MantenimientoIndicesInteligente')
BEGIN
    EXEC sp_delete_job @job_name = 'MantenimientoIndicesInteligente';
END
GO

EXEC sp_add_job
    @job_name = 'MantenimientoIndicesInteligente',
    @enabled = 1,

```

```

        @description = 'Mantenimiento inteligente de índices: REORGANIZE <30%,
REBUILD >30%, actualizar estadísticas',
        @start_step_id = 1,
        @category_name = 'Database Maintenance',
        @owner_login_name = 'sa';
GO

EXEC sp_add_jobstep
    @job_name = 'MantenimientoIndicesInteligente',
    @step_id = 1,
    @step_name = 'Ejecutar mantenimiento inteligente',
    @command = 'EXEC
GradosTitulos.Administracion.SP_MantenimientoIndicesInteligente
@UpdateStatistics = 1;',
    @database_name = 'GradosTitulos',
    @subsystem = 'TSQL',
    @retry_attempts = 2,
    @retry_interval = 15;
GO

-- Programación: Sábados a las 04:00 AM
EXEC sp_add_jobschedule
    @job_name = 'MantenimientoIndicesInteligente',
    @name = 'Semanal_Sabado_4AM',
    @freq_type = 8,
    @freq_interval = 7, -- Sábado
    @active_start_time = 040000;
GO

EXEC sp_add_jobserver
    @job_name = 'MantenimientoIndicesInteligente',
    @server_name = '(local)';
GO

PRINT 'Job "MantenimientoIndicesInteligente" creado.';

GO

```

Proyecto 4: Enunciado Automatizar el mantenimiento completo de la base de datos incluyendo:

Codigo:

```

-- =====
-- PROYECTO 4: Mantenimiento completo de base de datos
-- Backup, CHECKDB, limpieza, índices y estadísticas
-- =====

USE GradosTitulos;
GO

-- 1. TABLA PARA REGISTRO DE MANTENIMIENTO COMPLETO
IF OBJECT_ID('Auditoria.MantenimientoCompleto', 'U') IS NULL
BEGIN
    CREATE TABLE Auditoria.MantenimientoCompleto (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        FechaInicio DATETIME2 DEFAULT SYSDATETIME(),
        FechaFin DATETIME2 NULL,
        Fase VARCHAR(30) NOT NULL, -- CHECKDB, BACKUP, INDICES, STATS,
        LIMPIEZA

```

```

        Estado VARCHAR(20) NOT NULL,
        Mensaje NVARCHAR(MAX) NULL,
        DuracionMs INT NULL
    );

    PRINT 'Tabla Auditoria.MantenimientoCompleto creada.';
END
GO

-- 2. PROCEDIMIENTO DE MANTENIMIENTO COMPLETO
CREATE OR ALTER PROCEDURE Administracion.SP_MantenimientoCompleto
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @StartTime DATETIME2 = SYSDATETIME();
    DECLARE @IdRegistro INT;
    DECLARE @ErrorMsg NVARCHAR(4000);
    DECLARE @FaseActual VARCHAR(30);

    BEGIN TRY
        -- =====
        -- FASE 1: CHECKDB (Verificación de integridad)
        -- =====
        SET @FaseActual = 'CHECKDB';
        INSERT INTO Auditoria.MantenimientoCompleto (Fase, Estado, Mensaje)
        VALUES (@FaseActual, 'EN_EJECUCION', 'Iniciando verificación de
integridad...');
        SET @IdRegistro = SCOPE_IDENTITY();

        DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;

        UPDATE Auditoria.MantenimientoCompleto
        SET Estado = 'EXITOSO', Mensaje = 'CHECKDB completado sin errores.'
        WHERE Id = @IdRegistro;

        -- =====
        -- FASE 2: Backup completo
        -- =====
        SET @FaseActual = 'BACKUP';
        INSERT INTO Auditoria.MantenimientoCompleto (Fase, Estado, Mensaje)
        VALUES (@FaseActual, 'EN_EJECUCION', 'Iniciando backup completo...');
        SET @IdRegistro = SCOPE_IDENTITY();

        EXEC Administracion.SP_BackupCompletoGradosTitulos;

        UPDATE Auditoria.MantenimientoCompleto
        SET Estado = 'EXITOSO', Mensaje = 'Backup completado.'
        WHERE Id = @IdRegistro;

        -- =====
        -- FASE 3: Mantenimiento de índices (inteligente)
        -- =====
        SET @FaseActual = 'INDICES';
        INSERT INTO Auditoria.MantenimientoCompleto (Fase, Estado, Mensaje)
        VALUES (@FaseActual, 'EN_EJECUCION', 'Iniciando mantenimiento de
índices...');
        SET @IdRegistro = SCOPE_IDENTITY();

        EXEC Administracion.SP_MantenimientoIndicesInteligente
            @ReorganizeThreshold = 5.0,
            @RebuildThreshold = 30.0,

```

```

        @MinPages = 100,
        @UpdateStatistics = 0; -- No actualizar estadísticas aún

UPDATE Auditoria.MantenimientoCompleto
SET Estado = 'EXITOSO', Mensaje = 'Mantenimiento de índices
completado.'
WHERE Id = @IdRegistro;

-- =====
-- FASE 4: Actualización de estadísticas (FULLSCAN)
-- =====
SET @FaseActual = 'STATS';
INSERT INTO Auditoria.MantenimientoCompleto (Fase, Estado, Mensaje)
VALUES (@FaseActual, 'EN_EJECUCION', 'Iniciando actualización de
estadísticas...');
SET @IdRegistro = SCOPE_IDENTITY();

-- Actualizar estadísticas de todas las tablas con FULLSCAN
DECLARE @TableName NVARCHAR(128);
DECLARE @StatsCommand NVARCHAR(500);
DECLARE @StatsCount INT = 0;

DECLARE table_cursor CURSOR FOR
    SELECT TABLE_SCHEMA + '.' + TABLE_NAME
    FROM INFORMATION_SCHEMA.TABLES
    WHERE TABLE_TYPE = 'BASE TABLE'
      AND TABLE_CATALOG = 'GradosTitulos'
      AND TABLE_SCHEMA IN ('Academico', 'Tramite', 'Evaluacion',
'Documento', 'Catalogo');

OPEN table_cursor;
FETCH NEXT FROM table_cursor INTO @TableName;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @StatsCommand = 'UPDATE STATISTICS [' + @TableName + '] WITH
FULLSCAN;';
    PRINT 'Actualizando estadísticas: ' + @TableName;
    EXEC sp_executesql @StatsCommand;
    SET @StatsCount = @StatsCount + 1;

    FETCH NEXT FROM table_cursor INTO @TableName;
END

CLOSE table_cursor;
DEALLOCATE table_cursor;

UPDATE Auditoria.MantenimientoCompleto
SET Estado = 'EXITOSO',
    Mensaje = 'Estadísticas actualizadas para ' + CAST(@StatsCount AS
VARCHAR) + ' tablas.'
WHERE Id = @IdRegistro;

-- =====
-- FASE 5: Limpieza de historial (mantener 30 días)
-- =====
SET @FaseActual = 'LIMPIEZA';
INSERT INTO Auditoria.MantenimientoCompleto (Fase, Estado, Mensaje)
VALUES (@FaseActual, 'EN_EJECUCION', 'Iniciando limpieza de
historial...');
SET @IdRegistro = SCOPE_IDENTITY();

```

```

-- Limpiar historial de auditoría > 30 días
DELETE FROM Auditoria.AuditoriaJobs
WHERE FechaEjecucion < DATEADD(DAY, -30, GETDATE());

DECLARE @DeletedAudit INT = @@ROWCOUNT;

-- Limpiar historial de mantenimiento > 30 días
DELETE FROM Auditoria.MantenimientoIndices
WHERE FechaEjecucion < DATEADD(DAY, -30, GETDATE());

DECLARE @DeletedIndex INT = @@ROWCOUNT;

-- Limpiar historial de backups > 30 días (solo registros, no archivos)
DELETE FROM Auditoria.HistorialBackups
WHERE FechaBackup < DATEADD(DAY, -30, GETDATE());

DECLARE @DeletedBackup INT = @@ROWCOUNT;

-- Limpiar logs de errores antiguos
DELETE FROM Administracion.LogErrores
WHERE Fecha < DATEADD(DAY, -30, GETDATE());

DECLARE @DeletedErrors INT = @@ROWCOUNT;

UPDATE Auditoria.MantenimientoCompleto
SET Estado = 'EXITOSO',
    Mensaje = 'Limpieza completada: ' +
        CAST(@DeletedAudit AS VARCHAR) + ' registros de auditoría,
' +
        CAST(@DeletedIndex AS VARCHAR) + ' registros de índices,
' +
        CAST(@DeletedBackup AS VARCHAR) + ' registros de backups,
' +
        CAST(@DeletedErrors AS VARCHAR) + ' registros de errores
eliminados.'
WHERE Id = @IdRegistro;

-- =====
-- FINAL: Resumen completo
-- =====
DECLARE @EndTime DATETIME2 = SYSDATETIME();
DECLARE @TotalDuracionMs INT = DATEDIFF(MILLISECOND, @StartTime,
@EndTime);

PRINT '=====';
PRINT 'MANTENIMIENTO COMPLETO FINALIZADO';
PRINT '=====';
PRINT 'Duración total: ' + CAST(@TotalDuracionMs / 1000 AS VARCHAR) + '
segundos';
PRINT '=====';

END TRY
BEGIN CATCH
SET @ErrorMsg = 'ERROR en fase ' + @FaseActual + ': ' + ERROR_MESSAGE()
+
        ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) + ')';

IF @IdRegistro IS NOT NULL
BEGIN
    UPDATE Auditoria.MantenimientoCompleto
    SET
        Estado = 'FALLIDO',

```

```

        Mensaje = @ErrorMsg,
        FechaFin = SYSDATETIME()
    WHERE Id = @IdRegistro;
END

    PRINT @ErrorMsg;
    THROW;
END CATCH

END
GO

-- 3. CREAM JOB
USE msdb;
GO

IF EXISTS (SELECT 1 FROM sysjobs WHERE name =
'MantenimientoCompletoGradosTitulos')
BEGIN
    EXEC sp_delete_job @job_name = 'MantenimientoCompletoGradosTitulos';
END
GO

EXEC sp_add_job
    @job_name = 'MantenimientoCompletoGradosTitulos',
    @enabled = 1,
    @description = 'Mantenimiento completo: CHECKDB, Backup, Índices,
Estadísticas, Limpieza',
    @start_step_id = 1,
    @category_name = 'Database Maintenance',
    @owner_login_name = 'sa';
GO

EXEC sp_add_jobstep
    @job_name = 'MantenimientoCompletoGradosTitulos',
    @step_id = 1,
    @step_name = 'Ejecutar mantenimiento completo',
    @command = 'EXEC GradosTitulos.Administracion.SP_MantenimientoCompleto;',
    @database_name = 'GradosTitulos',
    @subsystem = 'TSQL',
    @retry_attempts = 2,
    @retry_interval = 30;
GO

-- Programación: Domingos a las 01:00 AM (después de otros mantenimientos)
EXEC sp_add_jobschedule
    @job_name = 'MantenimientoCompletoGradosTitulos',
    @name = 'Semanal_Domingo_1AM',
    @freq_type = 8,
    @freq_interval = 1,  -- Domingo
    @active_start_time = 010000;
GO

EXEC sp_add_jobserver
    @job_name = 'MantenimientoCompletoGradosTitulos',
    @server_name = '(local)';
GO

PRINT 'Job "MantenimientoCompletoGradosTitulos" creado.';
PRINT 'Programación: Domingos a las 01:00 AM.';

GO

```


Proyecto 5: Enunciado Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.

Codigo:

--

=====

-- PROYECTO 5: Plan inteligente de mantenimiento con cursores y DMV

-- Análisis automático y ejecución según fragmentación

--

=====

USE GradosTitulos;

GO

-- 1. TABLA DE ESTADÍSTICAS INTELIGENTES

IF OBJECT_ID('Auditoria.EstadisticasMantenimientoInteligente', 'U') IS NULL

BEGIN

CREATE TABLE Auditoria.EstadisticasMantenimientoInteligente (

Id INT IDENTITY(1,1) PRIMARY KEY,

FechaEjecucion DATETIME2 DEFAULT SYSDATETIME(),

TotalTablas INT NOT NULL,

TotalIndices INT NOT NULL,

TotalReorganizados INT NOT NULL,

TotalReconstruidos INT NOT NULL,

TotalEstadisticas INT NOT NULL,

TotalPaginasProcesadas BIGINT NOT NULL,

DuracionMs INT NOT NULL,

Detalles XML NULL,

```

    Resumen NVARCHAR(MAX) NULL

);

PRINT 'Tabla Auditoria.EstadisticasMantenimientoInteligente creada.';

END

GO

-- 2. PROCEDIMIENTO INTELIGENTE AVANZADO

CREATE OR ALTER PROCEDURE
Administracion.SP_MantenimientoInteligenteAvanzado

    @ReorganizeThreshold DECIMAL(5,2) = 5.0,

    @RebuildThreshold DECIMAL(5,2) = 30.0,

    @MinPages INT = 50,

    @UpdateStatistics BIT = 1,

    @EspecificoTablas NVARCHAR(MAX) = NULL -- Lista de tablas separadas por
coma

AS

BEGIN

    SET NOCOUNT ON;

    DECLARE @StartTime DATETIME2 = SYSDATETIME();

    DECLARE @TotalReorganizados INT = 0;

    DECLARE @TotalReconstruidos INT = 0;

    DECLARE @TotalEstadisticas INT = 0;

    DECLARE @TotalPaginas BIGINT = 0;

    DECLARE @TotalTablas INT = 0;

```

```
DECLARE @TotalIndices INT = 0;

DECLARE @ErrorMsg NVARCHAR(4000);

DECLARE @IdEstadistica INT;

DECLARE @DetallesXML XML;
```

-- Tabla para resultados

```
CREATE TABLE #Resultados (

    SchemaName NVARCHAR(128),

    TableName NVARCHAR(128),

    IndexName NVARCHAR(128),

    FragmentationAntes DECIMAL(5,2),

    FragmentationDespues DECIMAL(5,2),

    OperationType VARCHAR(20),

    Status VARCHAR(20),

    PagesProcessed INT

);
```

-- Tabla temporal para índices a procesar

```
CREATE TABLE #IndicesProcesar (

    Id INT IDENTITY(1,1) PRIMARY KEY,

    SchemaName NVARCHAR(128),

    TableName NVARCHAR(128),

    IndexName NVARCHAR(128),

    IndexID INT,

    Fragmentation DECIMAL(5,2),
```

```

    PageCount INT,

    OperationType VARCHAR(20)

);

BEGIN TRY

    -- Construir filtro de tablas si se especificó

    DECLARE @TableFilter NVARCHAR(MAX) = "";

    IF @EspecificoTablas IS NOT NULL

    BEGIN

        SET @TableFilter = ' AND OBJECT_NAME(ips.object_id) IN (' +
        @EspecificoTablas + ')';

    END

    -- Obtener índices con fragmentación

    DECLARE @Sql NVARCHAR(MAX) = '

    INSERT INTO #IndicesProcesar (

        SchemaName, TableName, IndexName, IndexID, Fragmentation,
        PageCount, OperationType

    )

    SELECT

        OBJECT_SCHEMA_NAME(ips.object_id) AS SchemaName,

        OBJECT_NAME(ips.object_id) AS TableName,

        i.name AS IndexName,

        ips.index_id AS IndexID,

        ips.avg_fragmentation_in_percent AS Fragmentation,

        ips.page_count AS PageCount,

```

```

CASE

    WHEN ips.avg_fragmentation_in_percent >= @RebuildThreshold
    THEN "REBUILD"

    WHEN ips.avg_fragmentation_in_percent >= @ReorganizeThreshold
    THEN "REORGANIZE"

    ELSE "NONE"

END AS OperationType

FROM sys.dm_db_index_physical_stats(DB_ID("GradosTitulos"), NULL,
NULL, NULL, "LIMITED") ips

JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
i.index_id

WHERE i.type > 0

AND ips.page_count > @MinPages

AND ips.avg_fragmentation_in_percent >= @ReorganizeThreshold

' + @TableFilter + '

ORDER BY ips.avg_fragmentation_in_percent DESC';

EXEC sp_executesql @Sql,

N'@ReorganizeThreshold DECIMAL(5,2), @RebuildThreshold
DECIMAL(5,2), @MinPages INT',

@ReorganizeThreshold, @RebuildThreshold, @MinPages;

-- Obtener estadísticas

SELECT @TotalTablas = COUNT(DISTINCT TableName),

@TotalIndices = COUNT(*)

FROM #IndicesProcesar;

-- Declarar cursor

```

```
DECLARE idx_cursor CURSOR FOR

    SELECT SchemaName, TableName, IndexName, IndexID, Fragmentation,
    PageCount, OperationType

    FROM #IndicesProcesar

    WHERE OperationType IN ('REORGANIZE', 'REBUILD')

    ORDER BY OperationType DESC, Fragmentation DESC;
```

```
DECLARE @SchemaName NVARCHAR(128);

DECLARE @TableName NVARCHAR(128);

DECLARE @IndexName NVARCHAR(128);

DECLARE @IndexID INT;

DECLARE @Fragmentation DECIMAL(5,2);

DECLARE @PageCount INT;

DECLARE @OperationType VARCHAR(20);

DECLARE @Command NVARCHAR(2000);

DECLARE @RegistroId INT;
```

```
OPEN idx_cursor;

FETCH NEXT FROM idx_cursor INTO @SchemaName, @TableName,
@IndexName, @IndexID,

    @Fragmentation, @PageCount, @OperationType;
```

```
WHILE @@FETCH_STATUS = 0

BEGIN

    SET @StartTime = SYSDATETIME();
```

```

BEGIN TRY

-- Registrar inicio en auditoría de índices

INSERT INTO Auditoria.MantenimientoIndices (

    Tabla, Indice, TipoOperacion, FragmentacionAntes, Estado

)

VALUES (

    @SchemaName + '.' + @TableName,

    @IndexName,

    @OperationType,

    @Fragmentation,

    'EN_EJECUCION'

);

SET @RegistroId = SCOPE_IDENTITY();

-- Construir comando

IF @OperationType = 'REORGANIZE'

BEGIN

    SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +

@SchemaName + '].[' + @TableName + '] REORGANIZE;';

    SET @TotalReorganizados = @TotalReorganizados + 1;

END

ELSE IF @OperationType = 'REBUILD'

BEGIN

    SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +

@SchemaName + '].[' + @TableName + '] REBUILD WITH (ONLINE = OFF,

SORT_IN_TEMPDB = ON);';

    SET @TotalReconstruidos = @TotalReconstruidos + 1;

```

END

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 120) + ']' +
@OperationType + ': ' + @SchemaName + '!' + @TableName + ' - ' + @IndexName;

EXEC sp_executesql @Command;

-- Actualizar estadísticas si está habilitado

IF @UpdateStatistics = 1

BEGIN

SET @Command = 'UPDATE STATISTICS [' + @SchemaName + '].[' +
@TableName + ']' [' + @IndexName + '] WITH FULLSCAN;;

EXEC sp_executesql @Command;

SET @TotalEstadisticas = @TotalEstadisticas + 1;

END

-- Obtener fragmentación después

DECLARE @FragDespues DECIMAL(5,2);

SELECT TOP 1 @FragDespues = avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'),

OBJECT_ID(@SchemaName + '.' + @TableName),

@IndexID, NULL, 'LIMITED');

-- Actualizar registro

UPDATE Auditoria.MantenimientoIndices

SET

FragmentacionDespues = @FragDespues,


```

        DuracionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME()),

        Estado = 'EXITOSO',

        Mensaje = 'Fragmentación: ' + CAST(@Fragmentation AS VARCHAR)
+ '% → ' +

        CAST(ISNULL(@FragDespues, 0) AS VARCHAR) + '% |
Páginas: ' + CAST(@PageCount AS VARCHAR)

        WHERE Id = @RegistroId;

-- Insertar en tabla de resultados

        INSERT INTO #Resultados (SchemaName, TableName, IndexName,
FragmentationAntes,

        FragmentationDespues, OperationType, Status,
PagesProcessed)

        VALUES (@SchemaName, @TableName, @IndexName, @Fragmentation,

        @FragDespues, @OperationType, 'EXITOSO', @PageCount);

        SET @TotalPaginas = @TotalPaginas + @PageCount;

END TRY

BEGIN CATCH

        SET @ErrorMsg = 'Error en ' + @OperationType + ' de ' + @IndexName +
': ' + ERROR_MESSAGE();

        IF @RegistroId IS NOT NULL

        BEGIN

                UPDATE Auditoria.MantenimientoIndices

                SET

```

```

        Estado = 'FALLIDO',

        Mensaje = @ErrorMsg,

        DuracionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME())

        WHERE Id = @RegistroId;

    END

    -- Insertar en tabla de resultados

    INSERT INTO #Resultados (SchemaName, TableName, IndexName,
FragmentationAntes,

        FragmentationDespues, OperationType, Status,
PagesProcessed)

    VALUES (@SchemaName, @TableName, @IndexName, @Fragmentation,

        NULL, @OperationType, 'FALLIDO', @PageCount);

    PRINT @ErrorMsg;

END CATCH

    FETCH NEXT FROM idx_cursor INTO @SchemaName, @TableName,
@IndexName, @IndexID,

        @Fragmentation, @PageCount, @OperationType;

END

CLOSE idx_cursor;

DEALLOCATE idx_cursor;

-- Generar XML con detalles

```

```

SET @DetallesXML = (
    SELECT
        SchemaName,
        TableName,
        IndexName,
        FragmentationAntes,
        FragmentationDespues,
        OperationType,
        Status,
        PagesProcessed
    FROM #Resultados
    FOR XML AUTO, ROOT('ResultadosMantenimiento')
);

```

-- Guardar estadísticas completas

```

INSERT INTO Auditoria.EstadisticasMantenimientoInteligente (
    TotalTablas,
    TotalIndices,
    TotalReorganizados,
    TotalReconstruidos,
    TotalEstadisticas,
    TotalPaginasProcesadas,
    DuracionMs,
    Detalles,
    Resumen

```

```

)

VALUES (

    @TotalTablas,

    @TotalIndices,

    @TotalReorganizados,

    @TotalReconstruidos,

    @TotalEstadisticas,

    @TotalPaginas,

    DATEDIFF(MILLISECOND,

        (SELECT MIN(FechaEjecucion) FROM Auditoria.MantenimientoIndices

            WHERE FechaEjecucion >= DATEADD(MINUTE, -5,
SYSDATETIME())),

        SYSDATETIME()),

    @DetallesXML,

    'Mantenimiento inteligente completado. ' +

    CAST(@TotalReorganizados AS VARCHAR) + ' reorganizados, ' +

    CAST(@TotalReconstruidos AS VARCHAR) + ' reconstruidos, ' +

    CAST(@TotalEstadisticas AS VARCHAR) + ' estadísticas actualizadas.'

);

-- Reporte final

PRINT '=====';

PRINT 'MANTENIMIENTO INTELIGENTE AVANZADO';

PRINT '=====';

PRINT 'Tablas procesadas: ' + CAST(@TotalTablas AS VARCHAR);

PRINT 'Índices procesados: ' + CAST(@TotalIndices AS VARCHAR);

```

```

PRINT 'REORGANIZADOS: ' + CAST(@TotalReorganizados AS VARCHAR);

PRINT 'REBUILT: ' + CAST(@TotalReconstruidos AS VARCHAR);

PRINT 'Estadísticas: ' + CAST(@TotalEstadisticas AS VARCHAR);

PRINT 'Páginas procesadas: ' + CAST(@TotalPaginas AS VARCHAR);

PRINT 'Duración: ' + CAST(DATEDIFF(SECOND, @StartTime,
SYSDATETIME()) AS VARCHAR) + ' segundos';

PRINT '=====';

DROP TABLE #IndicesProcesar;

DROP TABLE #Resultados;


END TRY

BEGIN CATCH

    SET @ErrorMsg = 'ERROR en mantenimiento inteligente: ' +
    ERROR_MESSAGE();

    PRINT @ErrorMsg;

    THROW;

END CATCH

END

GO


-- 3. CREAR JOB

USE msdb;

GO

IF EXISTS (SELECT 1 FROM sysjobs WHERE name =
'MantenimientoInteligenteAvanzado')

```

BEGIN

EXEC sp_delete_job @job_name = 'MantenimientoInteligenteAvanzado';

END

GO

EXEC sp_add_job

@job_name = 'MantenimientoInteligenteAvanzado',

@enabled = 1,

@description = 'Plan inteligente avanzado con cursores y DMV',

@start_step_id = 1,

@category_name = 'Database Maintenance',

@owner_login_name = 'sa';

GO

EXEC sp_add_jobstep

@job_name = 'MantenimientoInteligenteAvanzado',

@step_id = 1,

@step_name = 'Ejecutar mantenimiento inteligente',

@command = 'EXEC
GradosTitulos.Administracion.SP_MantenimientoInteligenteAvanzado
@ReorganizeThreshold = 5.0, @RebuildThreshold = 30.0, @MinPages = 100,
@UpdateStatistics = 1;';

@database_name = 'GradosTitulos',

@subsystem = 'TSQL',

@retry_attempts = 2,

@retry_interval = 15;

GO

-- Programación: Jueves a las 03:00 AM

EXEC sp_add_jobschedule

 @job_name = 'MantenimientoInteligenteAvanzado',

 @name = 'Semanal_Jueves_3AM',

 @freq_type = 8,

 @freq_interval = 5, -- Jueves

 @active_start_time = 030000;

GO

EXEC sp_add_jobserver

 @job_name = 'MantenimientoInteligenteAvanzado',

 @server_name = '(local)';

GO

PRINT 'Job "MantenimientoInteligenteAvanzado" creado.';

GO

TEMA 3. Automatización con T-SQL y PowerShell

Proyecto 1: Enunciado Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell.

Codigo:

```
-- =====
-- PROYECTO 1: Backup automático con T-SQL y PowerShell
-- Procedimiento almacenado + PowerShell Invoke-Sqlcmd
-- =====

USE GradosTitulos;
GO

-- 1. PROCEDIMIENTO ALMACENADO PARA BACKUP CON PARÁMETROS
CREATE OR ALTER PROCEDURE Administracion.SP_BackupAutomatico
    @RutaBackup NVARCHAR(500) = NULL,
    @NombreBackup NVARCHAR(200) = NULL,
    @Compression BIT = 1,
    @Verificar BIT = 1,
    @IdBackup INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @BackupPath NVARCHAR(500);
    DECLARE @BackupName NVARCHAR(200);
    DECLARE @FechaStr VARCHAR(20);
    DECLARE @ErrorMsg NVARCHAR(4000);
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
    DECLARE @TamanoMB DECIMAL(18,2);
    DECLARE @DuracionSeg INT;
    DECLARE @Cmd NVARCHAR(1000);

    BEGIN TRY
        -- Validar y establecer parámetros por defecto
        IF @RutaBackup IS NULL
            SET @RutaBackup = 'D:\Backups\GradosTitulos\';

        -- Asegurar que la ruta termina con \
        IF RIGHT(@RutaBackup, 1) != '\'
            SET @RutaBackup = @RutaBackup + '\';

        -- Generar nombre de backup
        SET @FechaStr = CONVERT(VARCHAR, GETDATE(), 112) + '_' +
            REPLACE(CONVERT(VARCHAR, GETDATE(), 108), ':', '');

        IF @NombreBackup IS NULL
            SET @NombreBackup = 'GradosTitulos_Backup_' + @FechaStr;

        SET @BackupPath = @RutaBackup + @NombreBackup + '.bak';

        -- Verificar que la carpeta existe
        DECLARE @FolderExists INT;
        EXEC master.dbo.xp_fileexist @RutaBackup, @FolderExists OUTPUT;

        IF @FolderExists = 0
            BEGIN
```



```

-- Intentar crear la carpeta
DECLARE @CreateFolderCmd NVARCHAR(1000);
SET @CreateFolderCmd = 'EXEC master.dbo.xp_create_subdir N''' +
@RutaBackup + '''';
EXEC sp_executesql @CreateFolderCmd;

PRINT 'Carpeta creada: ' + @RutaBackup;
END

-- Registrar inicio en auditoría
INSERT INTO Auditoria.HistorialBackups (
    TipoBackup,
    NombreArchivo,
    Estado,
    Compresion,
    Verificado
)
VALUES (
    'COMPLETO_AUTOMATICO',
    @BackupPath,
    'EN_EJECUCION',
    @Compresion,
    0
);

SET @IdBackup = SCOPE_IDENTITY();

-- Construir comando de backup
SET @Cmd = 'BACKUP DATABASE [GradosTitulos] TO DISK = N''' +
@BackupPath + ''' WITH ';

IF @Compresion = 1
    SET @Cmd = @Cmd + 'COMPRESSION, ';
ELSE
    SET @Cmd = @Cmd + 'NO_COMPRESSION, ';

SET @Cmd = @Cmd + 'NAME = N''' + @NombreBackup + ''', ';
SET @Cmd = @Cmd + 'DESCRIPTION = N''Backup automático - ' +
CONVERT(VARCHAR, GETDATE(), 120) + ''', ';
SET @Cmd = @Cmd + 'STATS = 10, ';

IF @Verificar = 1
    SET @Cmd = @Cmd + 'CHECKSUM, ';

SET @Cmd = @Cmd + 'INIT, SKIP, NOREWIND, NOUNLOAD, FORMAT;';

PRINT 'Ejecutando backup: ' + @Cmd;

-- Ejecutar backup
EXEC sp_executesql @Cmd;

-- Verificar el backup si está habilitado
IF @Verificar = 1
BEGIN
    SET @Cmd = 'RESTORE VERIFYONLY FROM DISK = N''' + @BackupPath + '''
WITH CHECKSUM, CONTINUE_AFTER_ERROR;';
    EXEC sp_executesql @Cmd;
END

-- Obtener tamaño del archivo
DECLARE @FileSize BIGINT;
EXEC master.dbo.xp_fileexist @BackupPath, @FileSize OUTPUT;

```

```

SET @TamanioMB = @FileSize / 1048576.0;

-- Calcular duración
SET @DuracionSeg = DATEDIFF(SECOND, @StartTime, SYSDATETIME());

-- Actualizar registro
UPDATE Auditoria.HistorialBackups
SET
    TamanioMB = @TamanioMB,
    DuracionSegundos = @DuracionSeg,
    Verificado = @Verificar,
    Estado = 'EXITOSO',
    Mensaje = 'Backup completado exitosamente. Tamaño: ' +
        CAST(@TamanioMB AS VARCHAR) + ' MB, Duración: ' +
        CAST(@DuracionSeg AS VARCHAR) + ' segundos'
WHERE Id = @IdBackup;

-- Devolver información
SELECT
    @IdBackup AS IdBackup,
    @BackupPath AS RutaArchivo,
    @TamanioMB AS TamanioMB,
    @DuracionSeg AS DuracionSegundos,
    'EXITOSO' AS Estado;

PRINT 'Backup completado: ' + @BackupPath;
PRINT 'Tamaño: ' + CAST(@TamanioMB AS VARCHAR) + ' MB';
PRINT 'Duración: ' + CAST(@DuracionSeg AS VARCHAR) + ' segundos';

END TRY
BEGIN CATCH
    SET @ErrorMsg = 'ERROR en backup: ' + ERROR_MESSAGE() +
        ' (Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) + ')';

    IF @IdBackup IS NOT NULL
    BEGIN
        UPDATE Auditoria.HistorialBackups
        SET
            Estado = 'FALLIDO',
            Mensaje = @ErrorMsg,
            DuracionSegundos = DATEDIFF(SECOND, @StartTime, SYSDATETIME())
        WHERE Id = @IdBackup;
    END

    PRINT @ErrorMsg;
    THROW;
END CATCH

END
GO

-- 2. PROCEDIMIENTO PARA OBTENER ESTADO DEL ÚLTIMO BACKUP
CREATE OR ALTER PROCEDURE Administracion.SP_ObtenerEstadoBackup
AS
BEGIN
    SET NOCOUNT ON;

    SELECT TOP 1
        Id,
        FechaBackup,
        TipoBackup,
        NombreArchivo,
        TamanioMB,

```

```

        DuracionSegundos,
        Compresion,
        Verificado,
        Estado,
        Mensaje
FROM Auditoria.HistorialBackups
ORDER BY FechaBackup DESC;
END

GO

```

Power shell:

```

# =====

# SCRIPT 1: Ejecutar Backup usando Invoke-Sqlcmd

# Guardar como: Execute-Backup.ps1

# =====

# Configuración

$ServerInstance = "localhost"

$Database = "GradosTitulos"

$BackupPath = "D:\Backups\GradosTitulos\"

$BackupName = "GradosTitulos_Backup_" + (Get-Date -Format "yyyyMMdd_HH:mm:ss")

# Crear carpeta si no existe

if (!(Test-Path $BackupPath)) {

    New-Item -ItemType Directory -Path $BackupPath -Force

    Write-Host "Carpeta creada: $BackupPath"

}

# Función para ejecutar el backup

function Invoke-BackupDatabase {

```

```

param(

    [string]$Server,

    [string]$Database,

    [string]$BackupPath,

    [string]$BackupName

)

Write-Host "===== " -ForegroundColor
Cyan

Write-Host "INICIANDO BACKUP AUTOMÁTICO" -ForegroundColor Cyan

Write-Host "===== " -ForegroundColor
Cyan

Write-Host "Servidor: $Server"

Write-Host "Base de datos: $Database"

Write-Host "Ruta: $BackupPath"

Write-Host "Nombre: $BackupName"

Write-Host "Fecha: $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')"
```

```

Write-Host "===== " -ForegroundColor
Cyan

try {

    # Construir la consulta SQL

    $sql = @"

DECLARE @IdBackup INT;

EXEC GradosTitulos.Administracion.SP_BackupAutomatico

    @RutaBackup = '$BackupPath',

    @NombreBackup = '$BackupName',

    @Compression = 1,

    @Verificar = 1,

    @IdBackup = @IdBackup OUTPUT;

```

```
SELECT @IdBackup AS IdBackup;
```

```
"@
```

```
Write-Host "Ejecutando procedimiento almacenado..." -ForegroundColor  
Yellow
```

```
# Ejecutar el backup
```

```
$result = Invoke-Sqlcmd -ServerInstance $Server -Database $Database -  
Query $sql
```

```
# Mostrar resultados
```

```
$idBackup = $result.IdBackup
```

```
Write-Host "Backup completado exitosamente!" -ForegroundColor Green
```

```
Write-Host "ID del Backup: $idBackup"
```

```
# Obtener detalles del backup
```

```
$sqlDetalles = @"
```

```
EXEC GradosTitulos.Administracion.SP_ObtenerEstadoBackup;
```

```
"@
```

```
$detalles = Invoke-Sqlcmd -ServerInstance $Server -Database $Database -  
Query $sqlDetalles
```

```
Write-Host "`nDETALLES DEL BACKUP:" -ForegroundColor Cyan
```

```
Write-Host "-----"
```

```
Write-Host "Archivo: $($detalles.NombreArchivo)"
```

```
Write-Host "Tamaño:  $::Round($detalles.TamanoMB, 2))$  MB"
```

```
Write-Host "Duración: $($detalles.DuracionSegundos) segundos"
```

```
Write-Host "Compresión: $(if($detalles.Compresion){'Sí'}else{'No'})"
```

```

Write-Host "Verificado: $(if($detalles.Verificado){'Sí'}else{'No'})"

Write-Host "Estado: $($detalles.Estado)"

Write-Host "-----"

# Verificar que el archivo existe

if (Test-Path $detalles.NombreArchivo) {

    $fileInfo = Get-Item $detalles.NombreArchivo

    Write-Host "✓ Archivo verificado: $($fileInfo.Name)" -
ForegroundColor Green

    Write-Host "  Tamaño real: $([math]::Round($fileInfo.Length / 1MB,
2)) MB"

    Write-Host "  Última modificación: $($fileInfo.LastWriteTime)"

} else {

    Write-Host "✗ Archivo no encontrado en la ruta especificada!" -
ForegroundColor Red

}

return $true

} catch {

    Write-Host "ERROR en el backup:" -ForegroundColor Red

    Write-Host $_.Exception.Message -ForegroundColor Red

    Write-Host $_.ScriptStackTrace -ForegroundColor Red

    return $false

}

}

# Ejecutar la función

$success = Invoke-BackupDatabase -Server $ServerInstance -Database $Database -
BackupPath $BackupPath -BackupName $BackupName

```

```
if ($success) {  
    Write-Host "`n===== " -ForegroundColor  
Green  
    Write-Host "✓ BACKUP COMPLETADO EXITOSAMENTE" -ForegroundColor Green  
    Write-Host "===== " -ForegroundColor  
Green  
} else {  
    Write-Host "`n===== " -ForegroundColor  
Red  
    Write-Host "✗ BACKUP FALLÓ" -ForegroundColor Red  
    Write-Host "===== " -ForegroundColor Red  
}
```

Proyecto 2: Enunciado Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.

Codigo:

```
=====
-- PROYECTO 2: Exportación diaria de trámites aprobados a CSV
-- =====

USE GradosTitulos;
GO

-- 1. VISTA PARA TRÁMITES APROBADOS
CREATE OR ALTER VIEW Administracion.V_TramitesAprobadosExport
AS
SELECT
    t.IdTramite,
    t.IdMatricula,
    t.FechaInicio,
    t.FechaUltMod,
    t.Observaciones,
    tt.Descripcion AS TipoTramite,
    et.Descripcion AS EstadoTramite,
    p.DNI,
    p.Nombres,
    p.ApellidoPaterno,
    p.ApellidoMaterno,
    p.Correo,
    p.Telefono,
    pr.NombrePrograma,
    pr.NumSemestres,
    pr.Modalidad,
    f.NombreFacultad,
    ti.IdTrabajo,
    ti.Titulo AS TituloTrabajo,
    ti.TipoTrabajo,
    ti.PorcentajeSimilitud,
    s.IdSustentacion,
    s.FechaHora AS FechaSustentacion,
    s.NotaExposicion,
    s.NotaDefensa,
    s.Condicion AS CondicionSustentacion,
    s.NroActa,
    d.NroDiploma,
    d.FechaExpedicion AS FechaDiploma,
    r.NroResolucion,
    r.FechaEmision AS FechaResolucion,
    COUNT(pg.IdPago) AS NumPagos,
    SUM(pg.Monto) AS TotalPagado
FROM Tramite.Tramite t
JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite
JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite = et.IdEstadoTramite
JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula
JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona
JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma
JOIN Academico.Facultad f ON pr.IdFacultad = f.IdFacultad
LEFT JOIN Tramite.TrabajoInvestigacion ti ON t.IdTramite = ti.IdTramite
LEFT JOIN Evaluacion.Sustentacion s ON ti.IdTrabajo = s.IdTrabajo AND
s.NroIntento = 1
LEFT JOIN Documento.Diploma d ON t.IdTramite = d.IdTramite
```



```

LEFT JOIN Documento.Resolucion r ON t.IdTramite = r.IdTramite AND
r.TipoResolucion = 'APROBACION_TITULO'
LEFT JOIN Tramite.Pago pg ON t.IdTramite = pg.IdTramite
WHERE et.Descripcion IN ('APROBADO', 'ARCHIVADO')
AND tt.Descripcion LIKE 'TITULO%'
GROUP BY
t.IdTramite, t.IdMatricula, t.FechaInicio, t.FechaUltMod, t.Observaciones,
tt.Descripcion, et.Descripcion, p.DNI, p.Nombres, p.ApellidoPaterno,
p.ApellidoMaterno, p.Correo, p.Telefono, pr.NombrePrograma,
pr.NumSemestres, pr.Modalidad, f.NombreFacultad, ti.IdTrabajo,
ti.Titulo, ti.TipoTrabajo, ti.PorcentajeSimilitud, s.IdSustentacion,
s.FechaHora, s.NotaExposicion, s.NotaDefensa, s.Condicion,
s.NroActa, d.NroDiploma, d.FechaExpedicion, r.NroResolucion, r.FechaEmision;
GO

```

-- 2. PROCEDIMIENTO PARA EXPORTAR TRÁMITES APROBADOS

```

CREATE OR ALTER PROCEDURE Administracion.SP_ExportarTramitesAprobados
@FechaDesde DATE = NULL,
@FechaHasta DATE = NULL,
@IncluirTodasLasColumnas BIT = 0
AS
BEGIN
    SET NOCOUNT ON;

    -- Si no se especifican fechas, usar el último mes
    IF @FechaDesde IS NULL
        SET @FechaDesde = DATEADD(MONTH, -1, GETDATE());

    IF @FechaHasta IS NULL
        SET @FechaHasta = GETDATE();

    -- Seleccionar datos
    IF @IncluirTodasLasColumnas = 1
    BEGIN
        SELECT *
        FROM Administracion.V_TramitesAprobadosExport
        WHERE CAST(FechaDiploma AS DATE) BETWEEN @FechaDesde AND @FechaHasta
            OR CAST(FechaResolucion AS DATE) BETWEEN @FechaDesde AND @FechaHasta
        ORDER BY FechaDiploma DESC, FechaResolucion DESC;
    END
    ELSE
    BEGIN
        -- Columnas principales para reporte
        SELECT
            IdTramite,
            DNI,
            ApellidoPaterno + ' ' + ApellidoMaterno + ', ' + Nombres AS
NombreCompleto,
            NombreFacultad,
            NombrePrograma,
            TipoTramite,
            FechaInicio,
            FechaSustentacion,
            CondicionSustentacion,
            NotaExposicion + NotaDefensa AS NotaFinal,
            NroDiploma,
            FechaDiploma,
            NroResolucion,
            FechaResolucion,
            TotalPagado
        FROM Administracion.V_TramitesAprobadosExport
        WHERE CAST(FechaDiploma AS DATE) BETWEEN @FechaDesde AND @FechaHasta
    END
END

```

```

        OR CAST(FechaResolucion AS DATE) BETWEEN @FechaDesde AND @FechaHasta
ORDER BY FechaDiploma DESC, FechaResolucion DESC;
END
END

GO

```

POWERSHELL:

```

# =====

# SCRIPT 1: Exportar trámites aprobados a CSV

# Guardar como: Export-TramitesAprobados.ps1

# =====

```

```

param(

    [string]$ServerInstance = "localhost",

    [string]$Database = "GradosTitulos",

    [string]$OutputPath = "D:\Reportes\GradosTitulos\",

    [string]$FechaDesde = (Get-Date).AddMonths(-1).ToString("yyyy-MM-dd"),

    [string]$FechaHasta = (Get-Date).ToString("yyyy-MM-dd")

)

```

```

# Crear carpeta si no existe

if (!(Test-Path $OutputPath)) {

    New-Item -ItemType Directory -Path $OutputPath -Force

    Write-Host "Carpeta creada: $OutputPath" -ForegroundColor Yellow

}

```

```

function Export-TramitesAprobados {

    param(

        [string]$Server,

```

```

        [string]$DB,

        [string]$FechaDesde,

        [string]$FechaHasta,

        [string]$Path

    )

    Write-Host "=====" -ForegroundColor
Cyan

    Write-Host "EXPORTACIÓN DE TRÁMITES APROBADOS" -ForegroundColor Cyan

    Write-Host "=====" -ForegroundColor
Cyan

    Write-Host "Servidor: $Server"

    Write-Host "Base de datos: $DB"

    Write-Host "Fecha desde: $FechaDesde"

    Write-Host "Fecha hasta: $FechaHasta"

    Write-Host "=====" -ForegroundColor
Cyan

    try {

        # Construir la consulta SQL

        $sql = @"

SET NOCOUNT ON;

EXEC GradosTitulos.Administracion.SP_ExportarTramitesAprobados

    @FechaDesde = '$FechaDesde',

    @FechaHasta = '$FechaHasta',

    @IncluirTodasLasColumnas = 0;

"@

        Write-Host "Consultando datos..." -ForegroundColor Yellow

```

```

# Ejecutar la consulta

$data = Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $sql

if ($data.Count -eq 0) {

    Write-Host "No se encontraron trámites aprobados en el período
especificado." -ForegroundColor Yellow

    return $false

}

Write-Host "Se encontraron $($data.Count) trámites aprobados." -
ForegroundColor Green

# Generar nombre del archivo

$fileName = "TramitesAprobados_" + (Get-Date -Format "yyyyMMdd_HHmss")
+ ".csv"

$fullPath = Join-Path -Path $Path -ChildPath $fileName

# Exportar a CSV

$data | Export-Csv -Path $fullPath -NoTypeInfo -Encoding UTF8

Write-Host "`nArchivo exportado exitosamente:" -ForegroundColor Green

Write-Host "  Ruta: $fullPath" -ForegroundColor Cyan

Write-Host "  Registros: $($data.Count)" -ForegroundColor Cyan

Write-Host "  Tamaño: $([math]::Round((Get-Item $fullPath).Length / 1KB,
2)) KB" -ForegroundColor Cyan

# Crear archivo con información adicional

$infoFile = $fullPath -replace "\.csv$", "_info.txt"

$infoContent = @"

```

INFORME DE EXPORTACIÓN

Fecha de exportación: \$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')

Base de datos: \$DB

Período: \$FechaDesde a \$FechaHasta

Total de registros: \$(\$data.Count)

Archivo: \$fileName

"@

\$infoContent | Out-File -FilePath \$infoFile -Encoding UTF8

return \$true

} catch {

Write-Host "ERROR en la exportación:" -ForegroundColor Red

Write-Host \$_.Exception.Message -ForegroundColor Red

Write-Host \$_.ScriptStackTrace -ForegroundColor Red

return \$false

}

}

Ejecutar la exportación

\$success = Export-TramitesAprobados -Server \$ServerInstance -DB \$Database -
FechaDesde \$FechaDesde -FechaHasta \$FechaHasta -Path \$OutputPath

if (\$success) {

Write-Host "`n=====" -ForegroundColor
Green

Write-Host "✓ EXPORTACIÓN COMPLETADA EXITOSAMENTE" -ForegroundColor Green

Write-Host "=====" -ForegroundColor
Green

} else {

```

Write-Host "`n===== " -ForegroundColor
Red

Write-Host "X EXPORTACIÓN FALLÓ" -ForegroundColor Red

Write-Host "===== " -ForegroundColor Red

}

```

Proyecto 3: Enunciado Desarrollar un proceso automático que:

- consulte diariamente la cantidad de trámites
- genere un archivo Exce
- almacene una copia en una carpeta de respaldo.

Codigo:

```

=====
-- PROYECTO 3: Exportación a Excel con reporte diario
-- =====

USE GradosTitulos;
GO

-- 1. PROCEDIMIENTO PARA REPORTE DIARIO DE TRÁMITES
CREATE OR ALTER PROCEDURE Administracion.SP_ReporteDiarioExcel
    @FechaReporte DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Si no se especifica fecha, usar hoy
    IF @FechaReporte IS NULL
        SET @FechaReporte = CAST(GETDATE() AS DATE);

    -- Reporte principal
    SELECT
        @FechaReporte AS FechaReporte,
        'RESUMEN GENERAL' AS Categoria,
        COUNT(*) AS Total,
        NULL AS Detalle
    FROM Trámite.Trámite
    WHERE CAST(FechaInicio AS DATE) = @FechaReporte

    UNION ALL

    SELECT
        @FechaReporte AS FechaReporte,
        'Por Tipo de Trámite' AS Categoria,
        COUNT(*) AS Total,
        tt.Descripcion AS Detalle
    FROM Trámite.Trámite t
    JOIN Trámite.TipoTrámite tt ON t.IdTipoTrámite = tt.IdTipoTrámite
    WHERE CAST(t.FechaInicio AS DATE) = @FechaReporte

```

```

GROUP BY tt.Descripcion

UNION ALL

SELECT
    @FechaReporte AS FechaReporte,
    'Por Estado' AS Categoria,
    COUNT(*) AS Total,
    et.Descripcion AS Detalle
FROM Tramite.Tramite t
JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite = et.IdEstadoTramite
WHERE CAST(t.FechaInicio AS DATE) = @FechaReporte
GROUP BY et.Descripcion

UNION ALL

SELECT
    @FechaReporte AS FechaReporte,
    'Por Facultad' AS Categoria,
    COUNT(*) AS Total,
    f.NombreFacultad AS Detalle
FROM Tramite.Tramite t
JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula
JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma
JOIN Academico.Facultad f ON pr.IdFacultad = f.IdFacultad
WHERE CAST(t.FechaInicio AS DATE) = @FechaReporte
GROUP BY f.NombreFacultad

ORDER BY Categoria, Total DESC;
END
GO

-- 2. PROCEDIMIENTO PARA REPORTE COMPLETO (DETALLADO)
CREATE OR ALTER PROCEDURE Administracion.SP_ReporteDiarioDetallado
    @FechaReporte DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF @FechaReporte IS NULL
        SET @FechaReporte = CAST(GETDATE() AS DATE);

    -- Detalle de trámites
    SELECT
        t.IdTramite,
        t.FechaInicio,
        t.FechaUltMod,
        tt.Descripcion AS TipoTramite,
        et.Descripcion AS Estado,
        p.DNI,
        p.Nombres + ' ' + p.ApellidoPaterno + ' ' + ISNULL(p.ApellidoMaterno,
        ''') AS NombreCompleto,
        pr.NombrePrograma,
        f.NombreFacultad,
        t.Observaciones,
        ti.Titulo AS TituloTrabajo,
        ti.PorcentajeSimilitud,
        s.Condicion AS CondicionSustentacion,
        d.NroDiploma,
        r.NroResolucion,
        (SELECT COUNT(*) FROM Tramite.Pago WHERE IdTramite = t.IdTramite) AS
        NumPagos,

```

```

        (SELECT SUM(Monto) FROM Trámite.Pago WHERE IdTrámite = t.IdTrámite) AS
TotalPagado
FROM Trámite.Trámite t
JOIN Trámite.TipoTrámite tt ON t.IdTipoTrámite = tt.IdTipoTrámite
JOIN Catalogo.EstadoTrámite et ON t.IdEstadoTrámite = et.IdEstadoTrámite
JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula
JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona
JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma
JOIN Academico.Facultad f ON pr.IdFacultad = f.IdFacultad
LEFT JOIN Trámite.TrabajoInvestigacion ti ON t.IdTrámite = ti.IdTrámite
LEFT JOIN Evaluacion.Sustentacion s ON ti.IdTrabajo = s.IdTrabajo AND
s.NroIntento = 1
LEFT JOIN Documento.Diploma d ON t.IdTrámite = d.IdTrámite
LEFT JOIN Documento.Resolucion r ON t.IdTrámite = r.IdTrámite AND
r.TipoResolucion = 'APROBACION_TITULO'
WHERE CAST(t.FechaInicio AS DATE) = @FechaReporte
ORDER BY t.FechaInicio;
END

GO

```

POWERSHELL:

```

# =====

# SCRIPT 2: Proceso completo con respaldo y notificación

# Guardar como: Complete-ExcelReport.ps1

# =====

param(

    [string]$ServerInstance = "localhost",

    [string]$Database = "GradosTitulos",

    [string]$OutputPath = "D:\Reportes\GradosTitulos\Excel\ ",

    [string]$BackupPath = "D:\Reportes\GradosTitulos\Excel\Backup\ ",

    [string]$EmailTo = "reportes@upla.edu.pe",

    [string]$SmtpServer = "smtp.upla.edu.pe"

)

# Importar módulo

```



```
Import-Module ImportExcel -Force
```

```
# Crear carpetas
```

```
if (!(Test-Path $OutputPath)) { New-Item -ItemType Directory -Path $OutputPath -  
Force }
```

```
if (!(Test-Path $BackupPath)) { New-Item -ItemType Directory -Path $BackupPath -  
Force }
```

```
Write-Host "===== " -  
ForegroundColor Cyan
```

```
Write-Host "PROCESO COMPLETO DE REPORTE EXCEL" -ForegroundColor  
Cyan
```

```
Write-Host "===== " -  
ForegroundColor Cyan
```

```
$startTime = Get-Date
```

```
$fechaReporte = (Get-Date).ToString("yyyy-MM-dd")
```

```
try {
```

```
    # 1. Generar reporte
```

```
    Write-Host "`n1. Generando reporte..." -ForegroundColor Yellow
```

```
    $sqlResumen = @"
```

```
EXEC GradosTitulos.Administracion.SP_ReporteDiarioExcel
```

```
    @FechaReporte = '$fechaReporte';
```

```
"@"
```

```
    $sqlDetalle = @"
```

EXEC GradosTitulos.Administracion.SP_ReporteDiarioDetallado

@FechaReporte = '\$fechaReporte';

"@

\$dataResumen = Invoke-Sqlcmd -ServerInstance \$ServerInstance -Database
\$Database -Query \$sqlResumen

\$dataDetalle = Invoke-Sqlcmd -ServerInstance \$ServerInstance -Database
\$Database -Query \$sqlDetalle

2. Crear archivo Excel

Write-Host "2. Creando archivo Excel..." -ForegroundColor Yellow

\$fileName = "Reporte_Tramites_" + (Get-Date -Format "yyyyMMdd") + ".xlsx"

\$fullPath = Join-Path -Path \$OutputPath -ChildPath \$fileName

Crear Excel con múltiples hojas

\$dataResumen | Export-Excel -Path \$fullPath -WorksheetName "Resumen" -
AutoSize \$true -TableStyle "Medium9"

\$dataDetalle | Export-Excel -Path \$fullPath -WorksheetName "Detalle" -AutoSize
\$true -TableStyle "Medium9" -Append

3. Respaldo del archivo

Write-Host "3. Realizando respaldo..." -ForegroundColor Yellow

\$backupFile = Join-Path -Path \$BackupPath -ChildPath \$fileName

Copy-Item -Path \$fullPath -Destination \$backupFile -Force

4. Mover archivos antiguos (> 30 días)

Write-Host "4. Limpiando archivos antiguos..." -ForegroundColor Yellow

```
$oldFiles = Get-ChildItem -Path $OutputPath -Filter "Reporte_Tramites_*.xlsx" |  
Where-Object {
```

```
    $_.LastWriteTime -lt (Get-Date).AddDays(-30)
```

```
}
```

```
foreach ($file in $oldFiles) {
```

```
    # Mover a backup
```

```
    $destFile = Join-Path -Path $BackupPath -ChildPath $file.Name
```

```
    Move-Item -Path $file.FullName -Destination $destFile -Force
```

```
    Write-Host "  Movido a backup: $($file.Name)" -ForegroundColor Green
```

```
}
```

5. Enviar correo (si se configuró)

```
if ($EmailTo) {
```

```
    Write-Host "5. Enviando notificación por correo..." -ForegroundColor Yellow
```

```
    $duration = (Get-Date) - $startTime
```

```
    $body = @"
```

REPORTE DIARIO DE TRÁMITES

Fecha del reporte: \$fechaReporte

Total de trámites: \$(\$dataDetalle.Count)

Archivo generado: \$fileName

Tamaño del archivo: $\$([math]::Round((Get-Item \$fullPath).Length / 1KB, 2))$ KB

Tiempo de generación: $\$([math]::Round(\$duration.TotalSeconds, 2))$ segundos

El archivo Excel se encuentra en:

$\$fullPath$

Resumen:

- Resumen de trámites: $\$($dataResumen.Count)$ categorías
- Detalle de trámites: $\$($dataDetalle.Count)$ registros

Adjunto: $\$fileName$

"@

$\$mailParams = @\{$

To = $\$EmailTo$

From = "reportes@upla.edu.pe"

Subject = "Reporte Diario de Trámites - $\$fechaReporte$ "

Body = $\$body$

SmtpServer = $\$SmtpServer$

Attachments = $@(\$fullPath)$

}

Send-MailMessage @mailParams

Write-Host " Correo enviado a: $\$EmailTo$ " -ForegroundColor Green

}

6. Guardar estadísticas

Write-Host "6. Guardando estadísticas..." -ForegroundColor Yellow

\$statsFile = Join-Path -Path \$OutputPath -ChildPath "Reporte_Stats.csv"

\$stats = [PSCustomObject]@{

 Fecha = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

 FechaReporte = \$fechaReporte

 TotalTramites = \$dataDetalle.Count

 TamanoKB = [math]::Round((Get-Item \$fullPath).Length / 1KB, 2)

 DuracionSeg = [math]::Round(\$duration.TotalSeconds, 2)

 Archivo = \$fileName

 EmailEnviado = \$true

}

if (Test-Path \$statsFile) {

 \$stats | Export-Csv -Path \$statsFile -NoTypeInfoInformation -Encoding UTF8 -
Append

} else {

 \$stats | Export-Csv -Path \$statsFile -NoTypeInfoInformation -Encoding UTF8

}

Write-Host "`n===== " -
ForegroundColor Green

Write-Host "✓ PROCESO COMPLETADO EXITOSAMENTE" -
ForegroundColor Green

```

Write-Host "===== " -
ForegroundColor Green

Write-Host " Archivo: $fileName"

Write-Host " Tamaño: $([math]::Round((Get-Item $fullPath).Length / 1KB, 2))
KB"

Write-Host " Registros: $($dataDetalle.Count)"

Write-Host " Duración: $([math]::Round($duration.TotalSeconds, 2)) segundos"

Write-Host "===== " -
ForegroundColor Green

} catch {

Write-Host "ERROR en el proceso:" -ForegroundColor Red

Write-Host $_.Exception.Message -ForegroundColor Red

Write-Host $_.ScriptStackTrace -ForegroundColor Red


# Enviar correo de error

if ($EmailTo) {

    $errorBody = @"

ERROR EN EL PROCESO DE REPORTE EXCEL


Fecha: $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')

Error: $($_.Exception.Message)


Stack Trace: $($_.ScriptStackTrace)

"@

    try {

```

```
Send-MailMessage -To $EmailTo -From "reportes@upla.edu.pe" -Subject  
"ERROR: Reporte de Trámites" -Body $errorBody -SmtpServer $SmtpServer
```

```
} catch {  
  
    # Silenciar error de email  
  
}  
  
}  
  
}
```